

Міністерство освіти і науки України
Національний університет «Запорізька політехніка»

МЕТОДИЧНІ ВКАЗІВКИ
до лабораторних робіт
з дисципліни
«ІНТЕЛЕКТУАЛЬНИЙ АНАЛІЗ ДАНИХ»

Методичні вказівки до лабораторних робіт з дисципліни
«Інтелектуальний аналіз даних» / Укл.: А. О. Олійник, С. О. Субботін,
Т. В. Федорончак, М. О. Андреев, В. В. Козлов – Запоріжжя:
НУ«Запорізька політехніка», 2023. – 109 с.

Укладачі: А. О. Олійник, д.т.н., професор
С. О. Субботін, д.т.н., професор
Т. В. Федорончак, к.т.н., доцент
М. О. Андреев, асистент
В. В. Козлов, студент

Рецензент: В. М. Льовкін, к.т.н., доцент

Відповідальний
за випуск: С. О. Субботін, д.т.н., професор

Затверджено
на засіданні кафедри
програмних засобів

Протокол № 1
від «17» серпня 2023 р.

ЗМІСТ

Зміст	3
Вступ	6
1 Лабораторна робота № 1 Програмне забезпечення інтелектуального аналізу даних	7
1.1 Мета роботи	7
1.2 Основні теоретичні відомості	7
1.2.1 Програма WEKA	9
1.2.2 Формат файлів даних, що використовується в WEKA	9
1.2.3 Мова програмування Python	11
1.2.4 Формат файлів даних, що використовується в Python	12
1.3 Завдання на лабораторну роботу	13
1.4 Контрольні питання	13
1.5 Зміст звіту	14
2 Лабораторна робота № 2 Основи попередньої обробки даних	15
2.1 Мета роботи	15
2.2 Основні теоретичні відомості	15
2.2.1 Відбір атрибутів	15
2.2.2 Пропущені значення	17
2.2.3 Нормалізація даних	17
2.2.4 Дискретизація числових атрибутів	19
2.2.5 Вибірка / семплювання (sampling)	19
2.2.6 Робота з програмою WEKA	20
2.2.7 Робота з Python	21
2.3 Завдання на лабораторну роботу	35
2.4 Контрольні питання	35
2.5 Зміст звіту	36

3 Лабораторна робота №3 Задача класифікації	37
3.1 Мета роботи	37
3.2 Основні теоретичні відомості.....	37
3.2.1 Метрики якості класифікаторів.....	38
3.2.2 Критерії порівняння роботи класифікаторів.....	44
3.2.3 Робота з програмою WEKA.....	44
3.2.4 Робота з Python.....	48
3.3 Завдання на лабораторну роботу.....	55
3.4 Контрольні питання.....	56
3.5 Зміст звіту.....	56
4 Лабораторна робота № 4 Прогнозування, задача регресії	57
4.1 Мета роботи	57
4.2 Основні теоретичні відомості.....	57
4.2.1 Метрики якості регресійних моделей.....	58
4.2.2 Робота з програмою WEKA.....	59
4.2.3 Робота з Python.....	60
4.3 Завдання на лабораторну роботу.....	66
4.4 Контрольні питання.....	67
4.5 Зміст звіту.....	67
5 Лабораторна робота № 5 Пошук асоціативних правил.....	68
5.1 Мета роботи	68
5.2 Основні теоретичні відомості.....	68
5.2.1 Метрики асоціативних правил	69
5.2.2 Робота з програмою WEKA.....	70
5.2.3 Робота з Python.....	72
5.3 Завдання на лабораторну роботу.....	76
5.4 Контрольні питання.....	77

5.5 Зміст звіту.....	77
6 Лабораторна робота № 6 Програмні засоби кластеризації даних.....	79
6.1 Мета роботи	79
6.2 Основні теоретичні відомості.....	79
6.2.1 Робота з програмою WEKA.....	80
6.2.2 Робота з Python.....	82
6.3 Завдання на лабораторну роботу.....	87
6.4 Контрольні питання.....	89
6.5 Зміст звіту.....	89
Перелік посилань	90
Додаток А Інтерфейс програми WEKA.....	91
А.1 Головне вікно програми	91
А.2 Модуль Explorer	92
А.2.1 Завантаження і попередня обробка даних (Preprocess).....	93
А.2.2 Класифікація (Classify).....	95
А.2.3 Кластеризація (Cluster).....	99
А.2.4 Асоціативні правила (Associate).....	100
А.2.5 Відбір атрибутів (Select attributes).....	100
А.2.6 Візуалізація (Visualize).....	102
А.3 Модуль Experimenter	102
А.4 Модуль Knowledge flow	104
А.5 Модуль Workbench та Simple CLI	105
Додаток В Варіанти індивідуальних завдань	106

ВСТУП

Метою практичної частини курсу «Інтелектуальний аналіз даних» є отримання практичних навичок з використання відповідних методів для вирішення практичних задач розробки програмного забезпечення для аналізу даних.

Протягом виконання лабораторних робіт студент має ознайомитися з принципами проведення інтелектуального аналізу даних за допомогою використання програмного забезпечення для аналізу даних WEKA та мови програмування Python з популярними бібліотеками для машинного навчання.

Відповідно до графіка студенти перед виконанням лабораторної роботи повинні ознайомитися з конспектом лекцій та рекомендованою літературою.

Для одержання заліку з кожної роботи студент здає викладачу оформлений звіт, а також демонструє на екрані комп'ютера результати виконання лабораторної роботи.

Звіт має містити:

- титульний аркуш;
- тему та мету роботи;
- завдання до роботи;
- лаконічний опис теоретичних відомостей;
- результати виконання лабораторної роботи;
- змістовний аналіз отриманих результатів та висновки.

Звіт виконують на білому папері формату A4 (210 × 297 мм) або подають в електронному вигляді.

Під час співбесіди при захисті лабораторної роботи студент повинний виявити знання про зміст роботи та методи виконання кожного етапу роботи, а також вміти продемонструвати результати роботи на конкретних прикладах. Студент повинний вміти правильно аналізувати отримані результати. Для самоперевірки при підготовці до виконання і захисту роботи студент повинен відповісти на контрольні запитання, наведені наприкінці опису відповідної роботи.

1 ЛАБОРАТОРНА РОБОТА № 1

ПРОГРАМНЕ ЗАБЕЗПЕЧЕННЯ ІНТЕЛЕКТУАЛЬНОГО АНАЛІЗУ ДАНИХ

1.1 Мета роботи

Ознайомитися та отримати навички роботи з графічним інтерфейсом користувача бібліотеки data mining алгоритмів WEKA та мовою програмування Python. Вивчити основні можливості програми WEKA та мови програмування Python.

1.2 Основні теоретичні відомості

Інтелектуальний аналіз даних (англ. data mining) – це збірна назва, що використовується для позначення сукупності методів виявлення в даних раніше невідомих, нетривіальних, практично корисних та доступних для інтерпретації знань, необхідних для прийняття рішень в різних сферах діяльності людини.

Необхідність інтелектуального аналізу даних виникла ще в кінці XX століття в результаті широкого поширення інформаційних технологій, що дозволяють детально протоколювати процеси бізнесу та виробництва.

За складом вирішуваних завдань data mining практично не відрізняється від стандартного набору засобів, що застосовуються в галузі прикладної статистики, машинного навчання (machine learning) та інформаційного пошуку (information retrieval).

Основна відмінність полягає в ефективності алгоритмів і технологічності їх застосування. Переважна більшість класичних процедур мають час виконання, квадратичний або навіть кубічний до обсягу вихідних даних. При кількості об'єктів, що перевищує кілька десятків тисяч, алгоритми працюють занадто повільно, навіть на сучасних комп'ютерах.

За останні десятиліття значні зусилля в цій області були спрямовані на створення спеціалізованих алгоритмів, здатних виконувати ті ж завдання за лінійний або навіть логарифмічний час без істотної втрати точності результату.

Глибинний аналіз даних здійснюється автоматично шляхом застосування методів математичної статистики, штучних нейронних

мереж, теорії нечітких множин або генетичних алгоритмів. Метою аналізу є виявлення правил та закономірностей, наприклад, статистичних подій. Так, наприклад, можливо виявити зміни у поведінці клієнтів або груп клієнтів для покращення стратегії підприємства.

Можна виділити такі основні задачі, що вирішують алгоритми інтелектуального аналізу даних: класифікація, регресія, кластеризація, пошук асоціативних правил.

Задача класифікації – формалізована задача, яка містить множину об'єктів (ситуацій), поділених певним чином на класи. Задана скінченна множина об'єктів (вибірка), для яких відомо, до яких класів вони належать. До якого класу належать інші об'єкти невідомо. Необхідно побудувати такий алгоритм, який буде здатний класифікувати довільний об'єкт з вихідної множини. Завдання кластеризації відноситься до статистичної обробки, а також до широкого класу завдань навчання з вчителем.

Задача регресії (регресійний аналіз) – розділ математичної статистики, присвячений методам аналізу залежності однієї величини від іншої. На відміну від кореляційного аналізу не з'ясовує чи істотний зв'язок, а займається пошуком моделі цього зв'язку, вираженої у функції регресії. Регресійний аналіз використовується в тому випадку, якщо відношення між змінними можуть бути виражені кількісно у виді деякої комбінації цих змінних. Отримана комбінація використовується для передбачення значення, що може приймати цільова (залежна) змінна, яка обчислюється на заданому наборі значень вхідних (незалежних) змінних.

Задача кластеризації – задача розбиття заданої вибірки об'єктів (ситуацій) на підмножини, які називаються кластерами, так, щоб кожен кластер складався зі схожих об'єктів, а об'єкти різних кластерів істотно відрізнялися. Завдання кластеризації відноситься до статистичної обробки, а також до широкого класу завдань навчання без вчителя.

Задача пошуку асоціативних правил – метод машинного навчання заснованого на правилах для знаходження цікавих відношень між змінними у великих базах даних. Метою є ідентифікація сильних правил, які виявляються в базах даних з використанням деяких вимірів зацікавленості.

Для вирішення наведених вище задач інтелектуального аналізу створюється спеціальне програмне забезпечення та відповідні

бібліотеки алгоритмів для різних мов програмування.

1.2.1 Програма WEKA

WEKA (Waikato Environment for Knowledge Analysis) – це бібліотека алгоритмів машинного навчання для вирішення завдань інтелектуального аналізу даних. Система дозволяє безпосередньо застосовувати алгоритми до вибірок даних, а також викликати алгоритми з програм написаних мовою Java.

WEKA – це продукт університету Уайкато (Нова Зеландія), який вперше був випущений в його сучасному вигляді в 1997 році. WEKA поширюється по ліцензії GNU General Public License (GNU GPL). Це програмне забезпечення написано на мові програмування Java та надає графічний користувацький інтерфейс для роботи з файлами даних і генерації візуальних результатів (у вигляді таблиць і графіків). Крім того є можливість інтегрувати WEKA, як і будь-яку іншу бібліотеку, у інші додатки, наприклад, для автоматизації аналізу даних на стороні сервера, використовуючи стандартний API.

Мета проекту – створити сучасне середовище для розробки методів машинного навчання та застосування їх до реальних даних, зробити методи машинного навчання доступними для повсюдного застосування. Передбачається, що за допомогою даного середовища фахівець прикладної області зможе використовувати методи машинного навчання для вилучення корисних знань безпосередньо з даних дуже великого обсягу.

Користувачами WEKA є дослідники в області машинного навчання і прикладних наук. Вона також широко використовується в навчальних цілях.

Опис основних можливостей графічного інтерфейсу користувача в програмі WEKA наведено в додатку А.

1.2.2 Формат файлів даних, що використовується в WEKA

Основний формат файлів даних, який використовується в програмі WEKA – це ARFF (Attribute-Relation File Format). В каталозі data, що міститься в папці, де встановлено програму, знаходяться приклади таких файлів.

ARFF файл – це ASCII текстовий файл, який описує список

об'єктів із загальними ознаками (атрибутами). Структурно такий файл розділено на дві частини: заголовок та дані.

В заголовку описується ім'я даних та їх метадані (імена атрибутів та їх типи). Наприклад:

```
% коментар
@relation myRelation
@attribute firstAttribute real
@attribute classAttribute {A,B}
```

В другій частині представляються самі дані. Наприклад:

```
@data
1.234, A
4.321, B
```

Заголовок містить інформацію про ім'я файлу й метадані даних. Ім'я відношення описується в такому форматі:

```
@relation <ім'я>
```

Іменем може бути будь-яка послідовність символів. Якщо ім'я містить пробіли, то воно має бути взято в лапки. Наприклад:

```
@relation Zaporizhzhia
@relation 'Zaporizhzhia polytechnic'
```

Метадані описують атрибути представлених у файлі даних. Інформація про кожний атрибут записується в окремому рядку і складається з імені атрибута та його типу. Всі імена атрибутів повинні бути унікальними. Порядок їх опису повинен збігатися з порядком колонок в описі самих даних. Загальний формат опису атрибута такий:

```
@attribute <ім'я атрибута> <тип атрибута>
```

Наприклад:

```
@attribute temperature date
```

Ім'я атрибута має починатися з символу «@». У разі якщо в імені містяться пробіли, воно має бути взято в лапки.

Поле <тип> може мати одне з таких значень:

- real;
- integer;
- <категорія>;
- string;
- date [<формат дати>].

Категоріальні типи описуються переліком категорій (можливих значень). Наприклад:

```
@attribute mark {excellent, good, average, bad}
```

Дані представляються в ARFF форматі у вигляді списку значень атрибутів об'єктів після тегу `@data`. Кожен рядок списку відповідає одному об'єкту, кожна колонка – атрибута, що описано в заголовку. В термінології data mining такі рядки називають векторами.

Дані можуть містити припущення (невідомі) значення. В ARFF файлах вони представляються символом «?», Наприклад:

```
@data
1.5, ?, 4.5, 'arithmetic progression'
```

Строкові дані, в тому випадку, якщо вони містять символи, що розділяють, повинні братися в лапки. Наприклад:

```
@relation universityName
@attribute short string
@attribute full string
```

```
@data
NUZP, 'National university Zaporizhzhia polytechnic'
```

При описі дати можна вказати формат, в якому вона записується. Дати також повинні братися в лапки.

```
@relation DateTime
@attribute datetime date "yyyy-MM-dd HH:mm:ss"
@data
"2020-01-01 12:00:00"
```

1.2.3 Мова програмування Python

Python – це одна з найпопулярніших високорівневих мультипарадигмальних мов програмування, що дозволяє створювати сценарії та програми, ґрунтуючись на принципах об'єктно-орієнтованого, функціонального та структурного програмування. Вона розроблена нідерландським програмістом Гвідо ван Россумом в якості простої мови для легкого вивчення новачком в програмуванні.

В наш час Python – широко поширена мова програмування, що використовується в багатьох областях діяльності людини: розробка прикладного програмного забезпечення та web застосунків, наукові розрахунки, машинне навчання та data mining, інше.

Однією з особливостей мови Python є те, що вона підтримує динамічну типізацію. Це означає, що на відміну від C++, C# та Java, де тип змінної визначається при її оголошенні, в Python тип змінної

тракується в залежності від присвоєного значення.

Для виконання лабораторних робіт рекомендується встановити Python версії 3.0 чи новіше.

Разом з інтерпретатором мови Python також постачається система керування пакетами `pip`. `Pip` – це система управління пакетами, яка використовується для установки й управління програмними пакетами, написаними на мові Python. Вона допомагає встановити бібліотеки, що будуть використовуватися при написанні програми.

Викликати функції системи управління пакетами можна за допомогою командного рядка Windows.

Для того, щоб дізнатися про всі команди системи та їх аргументи, необхідно виконати таку команду:

```
pip
```

Для встановлення пакетів необхідно використати функцію `install`:

```
pip install <назва пакета>
```

Після встановлення модулів та пакетів їх можна використати в своїй програмі. Для цього їх необхідно імпортувати в коді програми. Зробити це можна за допомогою команди `import`:

```
import <назва пакета>
```

В такому випадку для того, щоб звернутися до певного атрибута, який визначено в модулі необхідно використовувати таку конструкцію:

```
<назва пакета>.<атрибут>
```

Для підключення певних атрибутів модуля використовується інструкція `from`. Для зручності, щоб не писати назви модуля, при звертанні до атрибута, можна підключити потрібний атрибут окремо:

```
from <назва пакета> import <атрибут>
```

Для більш детального ознайомлення з основними можливостями мови програмування Python рекомендується пройти безкоштовні онлайн курси від компанії Python Institute «PCAP: Programming Essentials in Python», які опубліковано на офіційному сайті Networking Academy by Cisco: <https://www.netacad.com/courses/programming/pcap-programming-essentials-python>

1.2.4 Формат файлів даних, що використовується в Python

Набір даних для проведення інтелектуального аналізу може

зберігатися в різних форматах файлів: .csv, .tsv, .xlsx та інших. Найголовніше, якщо необхідно аналізувати дані за допомогою засобів мови програмування Python, потрібна структура даних, сумісна з ним.

Для виконання завдань на мові Python рекомендовано використовувати дані, що представлені в форматі CSV.

CSV формат (від англ. comma-separated values) – це файловий формат, який призначений для представлення табличних даних, поля яких відокремлюються символом коми та символом переходу на новий рядок. Таким чином рядок таблиці відповідає рядку тексту, який містить одне або кілька полів, розділених комами. Поля, що містять коми, лапки, або декілька рядків, мають обмежуватися з обох боків лапками.

В першому рядку такого файлу розташовують найменування стовпців таблиці, а в інших рядках – самі дані. Також дані можуть містити припущення (невідомі) значення. В CSV файлах такі дані просто пропускаються.

Приклад змісту файлу даних в CSV форматі:

```
Rank,Name,Platform,Year,Genre,Publisher
1,Wii Sports,Wii,2006,Sports,,
2,Super Mario Bros.,,1985,Platform,Nintendo
3,Mario Kart Wii,Wii,2008,,Nintendo
```

1.3 Завдання на лабораторну роботу

1. Ознайомитися з основними теоретичними відомостями за темою роботи, використовуючи ці методичні вказівки, а також рекомендовану літературу.

2. Встановити програму для інтелектуального аналізу даних WEKA та інтерпретатор мови програмування Python.

3. Ознайомитися з основними можливостями графічного інтерфейсу користувача програми WEKA та основами програмування на мові Python.

4. Відповісти на 10 контрольних запитань.

5. Оформити звіт з роботи.

1.4 Контрольні питання

1. Що таке інтелектуальний аналіз даних?

2. Які основні задачі вирішує інтелектуальний аналіз даних?

3. Що таке задача класифікації?
4. Що таке задача регресії?
5. Що таке задача кластеризації?
6. Що таке задача пошуку асоціативних правил?
7. Для чого використовується програма WEKA, які її можливості?
8. Яке призначення модулів Explorer, Experimenter, Knowledge flow, Workbench та SimpleCLI?
9. Опишіть призначення таких вкладок модуля Explorer: Preprocess, Classify, Cluster, Associate, Select Attributes, Visualize.
10. Опишіть формат файлів даних, який використовується в програмі WEKA.
11. Як завантажити дані для аналізу до програми WEKA?
12. Які основні властивості мови програмування Python?
13. Які парадигми програмування підтримує мова Python?
14. Які типи даних підтримує мова Python?
15. Які команди `pip` ви знаєте?
16. Як встановити модуль чи пакет?
17. Що необхідно зробити, щоб використати встановлену бібліотеку в коді програми за допомогою мови Python?
18. Опишіть формат файлів даних, який буде використовуватися при роботі з Python.

1.5 Зміст звіту

Звіт має складатися з таких розділів:

1. Тема і мета роботи;
2. Завдання до роботи;
3. Результати виконання завдань;
4. Відповіді на контрольні запитання;
5. Висновки, що відображають отримані результати роботи та їх критичний аналіз.

2 ЛАБОРАТОРНА РОБОТА № 2 ОСНОВИ ПОПЕРЕДНЬОЇ ОБРОБКИ ДАНИХ

2.1 Мета роботи

Ознайомитися та отримати навички роботи з програмою WEKA та бібліотеками мови програмування Python для проведення аналізу даних. На практиці вивчити методи попередньої обробки даних для задач інтелектуального аналізу даних.

2.2 Основні теоретичні відомості

Дані володіють такою характеристикою як якість, що містить такі параметри: точність, повнота, несуперечність, своєчасність, достовірність та інтерпретованість.

Для підвищення якості даних та підготовки їх до обробки методами інтелектуального аналізу існує декілька технологій попередньої обробки даних.

До основних задач попередньої обробки даних відносяться такі.

Задача очищення даних, яка використовується для заповнення пропущених значень, видалення шумів, видалення суперечливості, ідентифікації та видалення викидів.

Задача інтеграції даних із різних джерел (баз даних, кубів даних, файлів) в одне узгоджене сховище. Ця задача передбачає об'єднання даних й усунення неузгодженостей, дублікатів та конфліктів.

Задача проріджування та стиснення даних використовується для зменшення розміру даних з мінімізацією втрати інформації. Ця задача включає зниження розмірності даних (відбір атрибутів) та чисельне зменшення (побудова математичних моделей для значень атрибутів).

Задача перетворення даних. До неї відносяться нормалізація, дискретизація, квантування, згладжування, агрегація та відображення даних за допомогою ядерних функцій.

Також до попередньої обробки даних можна віднести перетворення задачі множинної класифікації в бінарну.

2.2.1 Відбір атрибутів

В більшості практичних ситуацій набори даних містять занадто

багато атрибутів, що збільшує час навчання алгоритмів. При цьому деякі з атрибутів є незначущими чи надмірними. Таким чином дані повинні бути попередньо оброблені з метою відбору деякої мінімальної підмножини атрибутів для навчання.

Для вибору хорошої підмножини атрибутів існує два підходи.

Перший з них заснований на незалежній оцінці статистичних чи інших характеристик набору даних. Такий підхід називається фільтрацією й відбувається до початку безпосереднього аналізу даних.

В другому підході відбір підмножини атрибутів виконується всередині методів інтелектуального аналізу. Такий підхід називається методом обгортки (*wrapper method*), тобто алгоритм навчання «обгорнутий» в процедуру відбору атрибутів.

Самі методи інтелектуального аналізу також можуть бути використані для відбору атрибутів. Наприклад, можна застосувати алгоритм побудови дерев рішень до повного набору даних і потім залишити в наборі тільки ті атрибути, які використані в побудованому дереві. Слід зауважити, що даний відбір атрибутів не дасть ніякого ефекту при побудові нового дерева, проте виявиться корисним при використанні інших методів аналізу.

Інша можливість – це застосувати до даних алгоритм, який будує лінійну модель (наприклад, метод опорних векторів), та ранжувати атрибути на підставі величин коефіцієнтів моделі. Атрибути з найменшими коефіцієнтами можуть бути відкинуті. Дану процедуру можна повторити декілька разів.

Крім того для відбору атрибутів можуть бути застосовані методи аналізу, що засновані на порівнянні близькості екземплярів вибірки. Для порівняння беруться сусідні примірники однакових та різних класів. Якщо в примірниках одного класу значення певного атрибута різні, то можна припустити, що даний атрибут є незначущим, а його вага повинна бути зменшена. З іншого боку, якщо в екземплярах різних класів атрибут має різні значення, то даний атрибут значимий і його вага повинна бути збільшена. Після декількох повторів даної процедури відбувається відбір атрибутів з найбільшими вагами. До недоліків даного методу можна віднести той факт, що даний метод не зможе визначити надлишкові атрибути, що пов'язані тісним кореляційним зв'язком.

Зазвичай пошук в просторі атрибутів відбувається в одному з двох напрямків: зверху вниз (починаючи з повного набору атрибутів і

відкидаючи на кожному кроці найгірший з них) або знизу вгору (починаючи з порожньої множини атрибутів і додаючи найкращий з решти)

В деяких випадках для поліпшення точності класифікації та кращого розуміння атрибутів для вирішення поставленого завдання можлива побудова нового атрибута на основі вже існуючих. Наприклад, можна ввести новий атрибут «Площа» на основі існуючих атрибутів «висота» й «ширина».

2.2.2 Пропущені значення

При роботі з даними, в яких є пропущені значення атрибутів для деяких екземплярів, існують такі стратегії поведінки:

- Відкинути екземпляри з пропущеними значеннями. Такий підхід застосовується насамперед для даних, у яких відсутнє значення цільового атрибута (для задач класифікації);
- Заповнити пропущені значення вручну;
- Застосувати глобальну константу (наприклад, «NaN», «None» або «Unknown»);
- Використати деяке статистично розраховане по всій вибірці значення (середнє арифметичне, медіану, моду);
- Використати статистичне значення, розраховане для примірників, що відносяться до того ж класу, як і розглянутий екземпляр;
- Використати найбільш ймовірне значення для атрибута. Це значення може бути розраховане за допомогою регресії, дерева рішень або інших математичних підходів.

2.2.3 Нормалізація даних

Одиниці виміру, що використовуються в деякому атрибуті, можуть вплинути на результати аналізу. Так, наприклад, перетворення одиниць вимірювання з метрів в дюйми для атрибута «висота» або перетворення з кілограмів у фунти для атрибута «вага» можуть призвести до різних результатів. В загальному випадку, вираз деякого атрибута в дрібніших одиницях виміру призведе до більш широкого діапазону значень для цього атрибута, що може призвести до більшої значущості або ж ваги даного атрибута.

Щоб уникнути залежності від вибору одиниць вимірювання та надати всім атрибутам однакову вагу дані повинні бути нормалізованими або нормованими. Нормалізація передбачає перетворення даних таким чином, щоб діапазон значень, що приймає атрибут, зменшився або став рівним $[-1; 1]$ або $[0; 1]$. Нормалізація найбільш ефективна в задачах із застосуванням нейронних мереж та задачах, алгоритми яких засновані на обчисленні відстаней.

Нехай у нас є числовий атрибут A з вимірними значеннями $a_1, a_2, \dots, a_n$.

Мінімаксна нормалізація. Нехай $\min_a$ – мінімальне значення атрибута, $\max_a$ – максимальне, а $[new_min_a; new_max_a]$ – новий діапазон для атрибута, тоді:

$$a'_i = \frac{a_i - \min_a}{\max_a - \min_a} (new_max_a - new_min_a) + new_min_a$$

Нормалізація з нульовим середнім. Значення атрибута нормалізуються за допомогою математичного очікування $\bar{a}$ та стандартного відхилення σ_a атрибута:

$$a'_i = \frac{a_i - \bar{a}}{\sigma_a}$$

Існує варіація нормалізації з нульовим середнім, в якому замість стандартного відхилення атрибута використовується середнє абсолютне значення атрибута:

$$S_a = \frac{1}{n} (|a_1 - \bar{a}| + |a_1 - \bar{a}| + \dots + |a_n - \bar{a}|)$$

Нормалізація за допомогою десяткової шкали:

$$a'_i = \frac{a_i}{10^j}$$

Де j – найменше ціле число, таке, що $(|a_i|) < 1$.

2.2.4 Дискретизація числових атрибутів

Дискретизація числових атрибутів є обов'язковою та необхідною в разі застосування алгоритмів інтелектуального аналізу, що працюють тільки з категоріальними атрибутами. Крім того, алгоритми, що працюють з числовими атрибутами часто дають кращі результати або ж працюють швидше, якщо значення атрибутів попередньо приведено до дискретної форми.

Методи дискретизації можуть бути класифіковані за двома параметрами:

- чи використовується в них інформація про класи: дискретизація з вчителем (supervised discretization) або дискретизація без вчителя (unsupervised discretization);

- в якому напрямку відбувається дискретизація: зверху-вниз (дискретизація починається з однієї або декількох точок поділу, а далі отримані інтервали рекурсивно розбиваються – метод розбиття) або знизу-вгору (спочатку всі значення атрибуту розглядаються як потенційні точки поділу, а далі сусідні значення рекурсивно об'єднуються, утворюючи інтервали – метод об'єднання).

2.2.5 Вибірка / семплювання (sampling)

Вибірка або семплювання застосовується в якості методу зменшення початкового набору даних з метою представлення великої вихідної множини екземплярів вибірки (генеральна сукупність) набагато меншою за розміром підмножиною (вибіркою з генеральної сукупності).

Генеральна сукупність – це вся множина однорідних за певною ознакою об'єктів чи подій, які є предметом інтересу або дослідження. Зазвичай, обсяг генеральної сукупності дуже великий, що робить прийняття до уваги всіх членів непрактичним або неможливим.

Вибірка з генеральної сукупності – це множина об'єктів або подій, вибраних за допомогою визначеної процедури з генеральної сукупності для участі в дослідженні. Вибірка являє собою множину або сукупність певного обсягу, члени якої збираються і статистичні характеристики обчислюються таким чином, що в результаті можна зробити висновки або екстраполяцію із вибірки на всю генеральну сукупність.

Припустимо, що вихідний набір даних D містить N примірників. Розглянемо найбільш загальні шляхи зменшення його розміру:

- Проста випадкова вибірка без повернення: з вихідного набору D випадковим чином вибирається S примірників ($S < N$), при цьому ймовірність вибору кожного примірника рівномірна.

- Проста випадкова вибірка з поверненням: схожа на попередню, однак з тією відмінністю, що після вибору примірника, він повертається у вихідну вибірку й згодом знову може бути вибраний.

- Кластерна вибірка: якщо вихідна вибірка згрупована в деякі роз'єднані «кластери» (наприклад, сторінки з бази даних або дані з різних географічних джерел), то до кожного з таких кластерів може бути застосована проста випадкова вибірка.

- Стратифікована вибірка: якщо вихідна вибірка несиметрична щодо розподілу класів і може бути розділена на страти, то проста випадкова вибірка застосовується до кожної страти окремо (наприклад, якщо дані представляють відомості про покупців різних вікових груп та при цьому кількість представників різних груп не однакова, такий підхід дозволить не втратити відомості про рідкісні групи покупців).

2.2.6 Робота з програмою WEKA

Для попередньої обробки даних за допомогою програми WEKA можна використати функції, що надає графічний інтерфейс користувача середовища Explorer, а саме: вкладки Preprocess (завантаження й попередня обробка даних) та Select attributes (відбір атрибутів).

Для попередньої обробки набору даних, що представлено у вибірці, необхідно скористатися можливостями, що надаються у вкладці Preprocess.

Спочатку необхідно завантажити файл з даними або згенерувати його за допомогою функцій пункту Generate. Далі за допомогою поля Filter можна застосувати різні алгоритми для редагування вибірки й виконати фільтрацію даних.

Для відбора найбільш значимих атрибутів необхідно скористатися можливостями, що надаються у вкладці Select attributes. Зробити це можна вказав в налаштуваннях тип оцінювача атрибутів та бажаний метод пошуку. В результаті виконання алгоритмів на екрані буде відображено результат відбору, проте незначимі атрибути з

вибірки видалено не буде (для видалення незначимих атрибутів необхідно скористатися фільтрами у вкладці Preprocess).

Більш детальний опис роботи з вкладками Preprocess та Select attributes графічного інтерфейсу користувача програми WEKA наведено в додатку А.

2.2.7 Робота з Python

В процесі аналізу й обробки даних виконуються деякі типові операції – завантаження даних в список або словник, очищення й фільтрація даних. Багато з цих операцій часто повторюються, виконуються за стандартними схемами, прості й часто одноманітні.

Перед початком роботи з мовою програмування Python необхідно підготувати набір даних. Для аналізу тих же самих даних, що використовувалися для data mining в WEKA необхідно перетворити файл ARFF формату до CSV формату. Зробити це можна за допомогою базових функцій мови програмування Python. Напишемо власний конвертер ARFF файлів до CSV файлів:

```
# Імпортуємо бібліотеку з функціями для взаємодії з ОС
import os

# Оголошення функції main
def main():

    # Отримуємо шлях до файлу
    file = input("Write filename: ")

    # Перевіряємо наявність файлу
    if os.path.exists(file):
        # Відкриваємо файл в режимі читання
        with open(file, "r") as InFile:
            # Зчитуємо всі дані з файлу в список
            file_content = InFile.readlines()
            # Зберігаємо ім'я файлу без розширення
            name, ext = os.path.splitext(InFile.name)

            # Позначка про початок набору даних
            data = False
            # Рядок з назвами атрибутів
            header = ""
            # Зміст відконвертованого файлу
            content = []
```

```

# Перебираємо всі рядки змісту вхідного ARFF файлу
for line in file_content:
# Пропускаємо якщо знайшли рядок з коментарем
    if line.strip(' ').startswith("%"):
        continue
# Інакше, поки не знайшли набір даних
    elif not data:
# Якщо знайшли атрибут
        if "@ATTRIBUTE" in line:
            attr = line.split()
# Отримуємо назву атрибута
            attr_name = attr[attr.index("@ATTRIBUTE") + 1]
# Записуємо назву атрибута до рядка з назвами
            header = header + attr_name + ", "
# Якщо знайшли набір даних
            elif "@DATA" in line:
# В позначці вказуємо, що знайшли набір даних
                data = True
# Видаляємо зайву кому в кінці рядка з назвами атрибутів
                header = header[:-1]
                header += '\n'
# Додаємо рядок з назвами атрибутів до змісту нового файлу
                content.append(header)
# Інакше, коли знайшли набір даних
            else:
# Записуємо рядок з даними до змісту й замінюємо невизначені
# дані пропуском
                content.append(line.replace("?", ""))

# Відкриваємо вихідний файл (якщо його нема він створиться)
with open(name + ".csv", "w") as OutFile:
# Записуємо зміст вихідного файлу
    OutFile.writelines(content)

# Перевірка відкриття коду безпосередньо й
# виклик функції main
if __name__ == "__main__":
    main()

```

Для попередньої обробки даних за допомогою мови програмування Python прийнято використовувати пакет *pandas* – один з інструментів обробки даних, що був створений для автоматизації рутинних завдань обробки даних і вже фактично став стандартом.

Пакет *pandas* – це програмна бібліотека, що призначена для обробки й аналізу даних. Він надає стандартну інфраструктуру для

зберігання даних та зручні засоби для часто виконуваних операцій. Робота pandas з даними будується поверх бібліотеки NumPy, що є інструментом нижчого рівня.

Пакет pandas робить Python потужним інструментом для аналізу даних. Пакет надає можливості для редагування й фільтрації даних, побудови зведених таблиць, виконання угруповань. Він надає зручний доступ до табличних даних, а при наявності пакета matplotlib надає можливість малювання графіків на отриманих наборах даних.

Для встановлення пакета pandas необхідно виконати таку команду:

```
pip install pandas
```

В результаті виконання цієї команди (рис. 2.1) буде встановлено пакет pandas та бібліотеки від яких він залежить :

- pandas;
- numpy – бібліотека, що надає можливості підтримки багатовимірних масивів та високорівневих математичних функцій, що призначені для роботи з багатовимірними масивами;
- python-dateutil – надає потужні розширення до стандартного модуля datetime, доступного в Python;
- pytz – забезпечує точні та крос-платформні розрахунки часового поясу;
- six – надає корисні функції для вирівнювання відмінностей між версіями Python з метою написання коду, сумісного для обох версій Python (2.x та 3.x).

```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.18362.836]
(c) Корпорация Майкрософт (Microsoft Corporation), 2019. Все права защищены.

C:\Users\Valeriy>pip install pandas
Collecting pandas
  Downloading pandas-1.0.4-cp37-cp37m-win32.whl (7.5 MB)
    |#####| 7.5 MB 6.4 MB/s
Collecting python-dateutil>=2.6.1
  Downloading python-dateutil-2.8.1-py2.py3-none-any.whl (227 kB)
    |#####| 227 kB 6.8 MB/s
Collecting pytz>=2017.2
  Downloading pytz-2020.1-py2.py3-none-any.whl (510 kB)
    |#####| 510 kB 3.2 MB/s
Collecting numpy>=1.13.3
  Downloading numpy-1.18.5-cp37-cp37m-win32.whl (10.8 MB)
    |#####| 10.8 MB 6.8 MB/s
Collecting six>=1.5
  Downloading six-1.15.0-py2.py3-none-any.whl (10 kB)
Installing collected packages: six, python-dateutil, pytz, numpy, pandas
Successfully installed numpy-1.18.5 pandas-1.0.4 python-dateutil-2.8.1 pytz-2020.1 six-1.15.0

C:\Users\Valeriy>
  
```

Рисунок 2.1 – Встановлення пакета pandas

Для візуалізації даних прийнято використовувати пакет matplotlib. Бібліотека pandas використовує саме його.

Пакет Matplotlib – це бібліотека на мові програмування Python для візуалізації даних за допомогою двовимірної графіки. Одержувані зображення й графіки можуть бути використані в якості ілюстрацій в різноманітних документах та публікаціях.

Для встановлення пакета pandas необхідно виконати таку команду:

```
pip install matplotlib
```

В результаті виконання цієї команди (рис. 2.2) буде встановлено пакет matplotlib та бібліотеки від яких він залежить :

- matplotlib;
- cycler – невід’ємна частина бібліотеки matplotlib;
- kiwisolver – це ефективна реалізація C ++ алгоритму вирішення обмежень Cassowary (інструментарій для вирішення поступових обмежень, який ефективно вирішує системи лінійних рівностей й нерівностей);
- pyparsing – це модуль що реалізує альтернативний підхід до створення та виконання простих граматик або використання регулярних виразів, порівняно з традиційним підходом;
- numpy;
- python-dateutil;
- six.

```

C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.18362.836]
(c) Корпорація Майкрософт (Microsoft Corporation), 2019. Все права захищені.

C:\Users\Valeriy>pip install matplotlib
Collecting matplotlib
  Downloading matplotlib-3.2.1-cp37-cp37m-win32.whl (9.0 MB)
    |#####| 9.0 MB 6.4 MB/s
Collecting kiwisolver>=1.0.1
  Downloading kiwisolver-1.2.0-cp37-none-win32.whl (44 kB)
    |#####| 44 kB 1.1 MB/s
Requirement already satisfied: python-dateutil>=2.1 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from matplotlib) (2.8.1)
Collecting cycler>=0.10
  Downloading cycler-0.10.0-py2.py3-none-any.whl (6.5 kB)
Collecting pyparsing!>=2.0.4,!>=2.1.2,!>=2.1.6,>=2.0.1
  Downloading pyparsing-2.4.7-py2.py3-none-any.whl (67 kB)
    |#####| 67 kB 2.8 MB/s
Requirement already satisfied: numpy>=1.11 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from matplotlib) (1.18.5)
Requirement already satisfied: six>=1.5 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from python-dateutil>=2.1->matplotlib) (1.15.0)
Installing collected packages: kiwisolver, cycler, pyparsing, matplotlib
Successfully installed cycler-0.10.0 kiwisolver-1.2.0 matplotlib-3.2.1 pyparsing-2.4.7

C:\Users\Valeriy>
  
```

Рисунок 2.2 – Встановлення пакета matplotlib

Щоб ефективно працювати з pandas, необхідно освоїти

найголовніші структури даних бібліотеки: DataFrame та Series. Без розуміння того, що вони з себе представляють, неможливо в подальшому проводити якісний аналіз.

Series (серія) – структура даних, що схожа на одновимірний масив (наприклад, list), але відмінною її рисою є наявність асоціативних міток (індексів) уздовж кожного елемента зі списку. Така особливість перетворює її в асоціативний масив або словник в Python.

Структура даних DataFrame (кадр даних) являє собою матрицю – двовимірну табличну структуру, схожу на таблицю реляційних баз даних, проте зберігається вона в оперативній пам'яті. В будь-якій таблиці завжди присутні рядки та стовпці. Стовпцями в об'єкті DataFrame виступають об'єкти Series, рядки яких є їхніми безпосередніми елементами.

Перед початком роботи імпортуємо необхідні бібліотеки:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

Для створення серії можна використати конструктор класу:

```
my_series = pd.Series(["First", "Second", "Third"])
print(my_series)
```

```
0    First
1    Second
2    Third
dtype: object
```

В строковому поданні об'єкта Series, індекс знаходиться ліворуч, а сам елемент праворуч. Якщо індекс явно не задано, то pandas автоматично створює RangeIndex від 0 до N-1, де N загальна кількість елементів. Також варто звернути увагу на те, що в серії є тип збережених елементів, в даному випадку це тип object.

Індекси об'єкта Series можуть бути не тільки чисельними, а й рядковими. Визначити їх можна при створенні об'єкта та / або змінити задав нове значення атрибуту index. Змінити тип даних, що зберігається в серії можна за допомогою функції astype().

```
my_series.index = ["one", "two", "three"]
my_series = my_series.astype("string")
print(my_series)
```

```
one    First
```

```
two      Second
three    Third
dtype: string
```

Доступ до елементів об'єкта Series здійснюється по їх індексу (згадується аналогія зі словником і доступом по ключу):

```
print(my_series["one"])
```

```
First
```

Альтернативний варіант доступу до елемента:

```
print(my_series.two)
```

```
Second
```

Щоб перший приклад роботи з кадром даних був якомога простіше, ми створимо для нього матрицю 3×3 . В Python така таблиця реалізується у вигляді списку списків:

```
my_table = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
print(my_table)
```

```
[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

На жаль, в Python така матриця зовсім не схожа на таблицю, якщо не вжити додаткових заходів. Подивимося, що можна зробити з цією ж таблицею як з кадром даних pandas:

```
my_dataframe = pd.DataFrame(my_table)
print(my_dataframe)
```

```
   0  1  2
0  1  2  3
1  4  5  6
2  7  8  9
```

Назви заголовків кадру даних також можна змінити:

```
my_dataframe.columns = ["one", "two", "three"]
print(my_dataframe)
```

```
   one  two  three
0    1    2     3
1    4    5     6
2    7    8     9
```

Щоб отримати список (серію), що зберігається в стовпці необхідно звернутися за індексом (нотація) або за атрибутом:

```
print(my_dataframe["two"])
```

```
0    2
1    5
2    8
```

```
print(my_dataframe.two)
```

```
0    2
1    5
2    8
```

Список стовпців в подвійних квадратних дужках дозволяє зробити більше – отримати підмножину кадра даних, яка теж є кадром даних. Створимо DataFrame з першим та третім стовпчиком:

```
my_dataframe = my_dataframe[["one", "three"]]
print(my_dataframe)
```

```
   one  three
0    1     3
1    4     6
2    7     9
```

Також сконструювати об'єкт DataFrame можна на основі вбудованого в Python типу даних Dictionary:

```
dict = {
    "city": ["Zaporizhzhia", "Dnipro", "Kyiv"],
    "population": [746749, 996400, 2884000],
    "square": [334, 405, 839]}
```

```
my_dataframe = pd.DataFrame(dict)
print(my_dataframe)
```

```
   city  population  square
0  Zaporizhzhia    746749    334
1      Dnipro    996400    405
2      Kyiv    2884000    839
```

Бібліотека pandas підтримує всі найпопулярніші формати зберігання даних: csv, excel, sql, буфер обміну, html та інші. Найчастіше доводиться працювати з CSV файлами. Зчитати дані з CSV файлу та перетворити їх в DataFrame можна за допомогою функції read_csv():

```
data = pd.read_csv("D:\\Data.csv")
print(data)
```

	height	weight	hair	hair_length	age	class
0	NaN	76	Brown	5	25	Man
1	1.70	83	Gray	3	55	Man
2	1.83	89	Brown	6	37	Man
3	1.73	70	Blond	7	42	Man
4	1.74	67	NaN	35	31	Man
5	1.60	63	Gray	35	57	Woman
6	NaN	65	Blond	30	51	Woman
7	1.65	70	NaN	55	37	Woman
8	1.62	60	Brown	75	20	Woman
9	1.72	75	Blond	40	35	Woman

В тих випадках, коли невідомі (пропущені) значення атрибутів визначені якимось іншим чином, необхідно в функції завантаження використати параметр `na_values`. Таким чином всі значення, що будуть співпадати зі значеннями вказаними в `na_values` буде замінено на NaN:

```
data = pd.read_csv("D:\\Data.csv", na_values = ["?"])
```

Таке перетворення може бути особливо корисним, якщо вам дістався один з тих файлів даних, в яких з якоїсь невідомої причини пропущені дані позначаються різними способами: NA, N / A, ? та інші. В такому випадку варто переглянути дані, дізнатися, які позначення використовуються, а потім перезавантажити файл, задавши значення параметра `na_values` для стандартизації всіх цих варіантів в NaN.

NaN (Not a Number) – значення яке використовується для позначення невідомих значень. Визначене в бібліотеці `pumpy`: `pumpy.nan`.

Перед вирішенням задач класифікації, регресії, кластеризації та пошуку асоціативних правил необхідно позбутися від таких значень, адже деякі алгоритми погано працюють з ними.

Зберегти кадр даних в файл можна за допомогою функції `to_csv()`:

```
data.to_csv("D:\\TestSave.csv")
```

До колонок можна звертатися використовуючи атрибут чи нотацію:

```
print(data["weight"]) # print(data.weight)
```

```
0    76
1    83
2    89
3    70
4    67
5    63
```

```

6    65
7    70
8    60
9    75

```

Звернутися до рядка даних можна за допомогою параметра `loc` структури `DataFrame`:

```

print(data.loc[1])

height    1.7
weight    83
hair_length  3
age       55
class     Man
Name: 1, dtype: object

```

До певного значення можна звернутися як до елементу двовимірного масиву:

```
print(data["weight"][0]) # print(data.weight[0])
```

```
76
```

Видалити не потрібну колонку чи рядок даних можна за допомогою функції `drop()`:

```

data = data.drop(["hair"], axis = 1)
print(data)

```

	height	weight	hair_length	age	class
0	NaN	76	5	25	Man
1	1.70	83	3	55	Man
2	1.83	89	6	37	Man
3	1.73	70	7	42	Man
4	1.74	67	35	31	Man
5	1.60	63	35	57	Woman
6	NaN	65	30	51	Woman
7	1.65	70	55	37	Woman
8	1.62	60	75	20	Woman
9	1.72	75	40	35	Woman

Параметр `axis` визначає що саме видаляти: колонку чи рядок. При значенні 0 видаляється рядок за визначеним індексом, а при значенні 1 – колонка.

Переглянути статистичний опис набору даних можна за допомогою функції `describe()`:

```
data.describe()
```

	height	weight	hair_length	age
count	8.000000	10.000000	10.000000	10.000000
mean	1.698750	71.800000	29.100000	39.000000
std	0.074342	9.077445	24.154365	12.373807
min	1.600000	60.000000	3.000000	20.000000
25%	1.642500	65.500000	6.250000	32.000000
50%	1.710000	70.000000	32.500000	37.000000
75%	1.732500	75.750000	38.750000	48.750000
max	1.830000	89.000000	75.000000	57.000000

Отримати інформацію про кадр даних можна за допомогою функції `info()`:

```
data.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   height          8 non-null     float64
1   weight          10 non-null    int64
2   hair_length     10 non-null    int64
3   age             10 non-null    int64
4   class           10 non-null    object
dtypes: float64(1), int64(3), object(1)
memory usage: 424.0+ bytes
```

Дізнатися скільки пропущених даних в вибірці можна за допомогою функцій `isnull()` та `sum()`:

```
data.isnull().sum()
height      2
weight      0
hair_length 0
age         0
class       0
dtype: int64
```

Для заміни пропущених даних можна використати функцію `fillna()`. Наприклад замінемо пропущені дані в колонках на середні значення (функція `mean()`):

```
print(data.fillna(data.mean()))
   height  weight  hair_length  age  class
0  1.69875    76           5    25    Man
1  1.70000    83           3    55    Man
2  1.83000    89           6    37    Man
```

3	1.73000	70	7	42	Man
4	1.74000	67	35	31	Man
5	1.60000	63	35	57	Woman
6	1.69875	65	30	51	Woman
7	1.65000	70	55	37	Woman
8	1.62000	60	75	20	Woman
9	1.72000	75	40	35	Woman

Проте іноді необхідно замінювати пропущені значення не на загальне середнє значення стовпчика, а на середнє значення з врахуванням класу.

Для цього спочатку виконаємо функцію групування даних й визначення середніх значень для кожного з класів:

```
mean_by_class = data.groupby("class").mean()
print(mean_by_class)
```

	height	weight	hair_length	age
class				
Man	1.7500	77.0	11.2	38.0
Woman	1.6475	66.6	47.0	40.0

Замінемо пропущені значення на середні з врахуванням класів:

```
data["height"] =
data["height"].fillna(data["class"].map(mean_by_class["height"]))
print(data)
```

	height	weight	hair_length	age	class
0	1.7500	76	5	25	Man
1	1.7000	83	3	55	Man
2	1.8300	89	6	37	Man
3	1.7300	70	7	42	Man
4	1.7400	67	35	31	Man
5	1.6000	63	35	57	Woman
6	1.6475	65	30	51	Woman
7	1.6500	70	55	37	Woman
8	1.6200	60	75	20	Woman
9	1.7200	75	40	35	Woman

Недоліком такого підходу є те, що для кожного атрибута необхідно виконати всі ці дії, адже функція map працює тільки з серією, а не з кадром даних.

Для зміни всіх невідомих даних в DataFrame з урахуванням класів можна використати функції groupby() та transform():

```
data = data.fillna(data.groupby("class").transform("mean"))
```

```
print(data)
```

	height	weight	hair_length	age	class
0	1.7500	76	5	25	Man
1	1.7000	83	3	55	Man
2	1.8300	89	6	37	Man
3	1.7300	70	7	42	Man
4	1.7400	67	35	31	Man
5	1.6000	63	35	57	Woman
6	1.6475	65	30	51	Woman
7	1.6500	70	55	37	Woman
8	1.6200	60	75	20	Woman
9	1.7200	75	40	35	Woman

Переглянути список типів даних, які було призначено для кожного з атрибутів можна за допомогою параметра `dtypes` в кадрі даних:

```
print(data.dtypes)
```

```
height      float64
weight      int64
hair_length int64
age         int64
class       object
dtype: object
```

Змінити призначений тип даних можна за допомогою функції `astype()`:

```
data["class"] = data["class"].astype("category")
print(data.dtypes)
```

```
height      float64
weight      int64
hair_length int64
age         int64
class       category
dtype: object
```

Візуалізовувати дані можна за допомогою можливостей бібліотеки `matplotlib`. `DataFrame` містить різні функції, в результаті виконання яких створюються об'єкти `pyplot` бібліотеки `matplotlib`. Для відображення графіка необхідно використати функцію `show()` для класа `pyplot`.

Побудуємо кругову діаграму, що відображає відносну кількість

класів:

```
data["class"].value_counts().plot(kind = "pie")
plt.show()
```

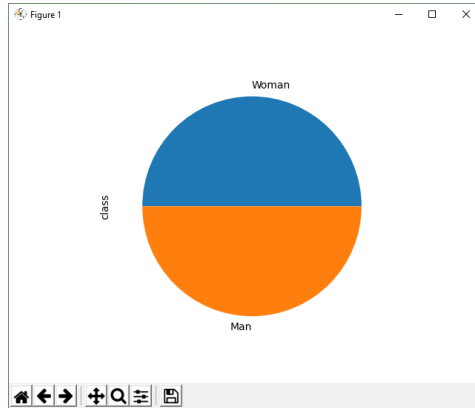


Рисунок 2.3 – Кругова діаграма

Побудуємо гістограму для різних атрибутів набору даних:

```
data.hist(edgecolor = "black", linewidth = 1)
plt.show()
```

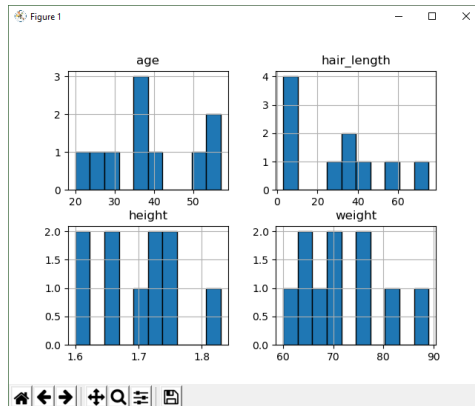


Рисунок 2.4 – Гістограма для різних атрибутів набору даних

Побудуємо графік залежності класів від атрибутів «weight» та «height»:

```
graph = data[data["class"]=="Man"].plot(kind = "scatter", x =
```

```

"weight", y = "height", color = "blue", label = "Man")
data[data["class"]=="Woman"].plot(kind = "scatter", x =
"weight", y = "height", color = "red", label = "Woman", ax = graph)
plt.show()

```

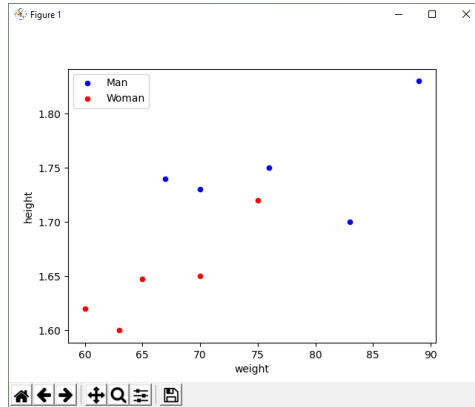


Рисунок 2.5 – Графік залежності класів від атрибутів «weight» та «height»

Побудуємо графік частот класів в залежності від значень атрибута «hair_length»:

```

tbl = data.groupby(["hair_length", "class"]).size().unstack()
tbl.plot(kind = "bar", stacked = True)
plt.show()

```

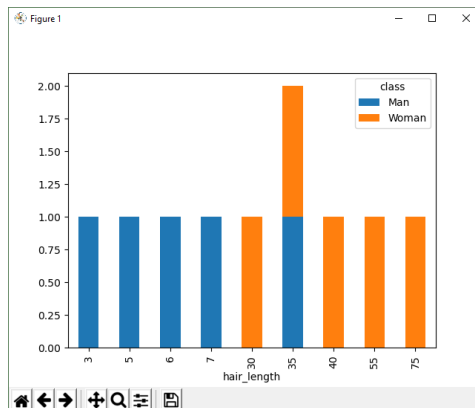


Рисунок 2.6 – Графік частот класів в залежності від значень атрибута «hair_length»

2.3 Завдання на лабораторну роботу

1. Обрати в додатку В вибірку для аналізу.

2. Виконати попередню обробку даних за допомогою програми WEKA:

- завантажити вибірку;
- описати яке практичне завдання вирішується;
- описати всі атрибути, які характеризують екземпляри вибірки, та їх типи даних;
- визначити, який атрибут є цільовим, які значення він приймає та скільки екземплярів кожного класу в вибірці;
- визначити чи є екземпляри з відсутніми значеннями. Якщо такі екземпляри є, то встановити їм відповідні значення використовуючи одну з наведених стратегії поведінки при роботі з пропущеними значеннями;
- визначити чи є викиди даних;
- подати дані в графічному вигляді.

3. Виконати попередню обробку даних за допомогою засобів мови програмування Python:

- конвертувати вибрану для аналізу вибірку з ARFF до CSV формату;
- використовуючи бібліотеку pandas виконати попередню обробку даних (завантажити файл, вивести на екран частину вибірки, для пропущених значень використати одну з наведених стратегій поведінки);
- використовуючи бібліотеку matplotlib побудувати необхідні графіки для подання даних в графічному вигляді (типи графіків: pie, hist, scatter, bar);

4. Відповісти на 5 контрольних запитань.

5. Оформити звіт з роботи.

2.4 Контрольні питання

1. Що таке попередня обробка даних?
2. Які задачі вирішує попередня обробка даних?
3. Що таке вибірка та генеральна сукупність?
4. Якими властивостями повинні володіти дані? Що означає термін репрезентативна вибірка?

5. Що таке якість даних? Яка мета підготовки даних до аналізу?
6. Що таке атрибут та екземпляр вибірки?
7. Який атрибут даних називають цільовим? Що таке значимий та незначимий атрибут?
8. Опишіть підходи для вибору хорошої підмножини атрибутів.
9. Опишіть стратегії поведінки при роботі з пропущеними значеннями атрибутів.
10. Що таке нормалізація даних та для чого вона використовується?
11. Що розуміють під фільтрацією в WEKA? В чому різниця між фільтрами атрибутів та фільтрами екземплярів? В чому різниця між unsupervised та supervised фільтрами?
12. Як вибрати й застосувати фільтр до даних в WEKA?
13. Для чого використовують пакет pandas?
14. Для чого використовують пакет Matplotlib?
15. Що таке структура даних Series та DataFrame? В чому різниця між ними?
16. Як завантажити набір даних за допомогою бібліотеки pandas?
17. Що означає значення NaN. В якій бібліотеці воно визначено?
18. За допомогою яких функцій бібліотеки pandas можна працювати з пропущеними даними?
19. Які функції бібліотеки pandas ви знаєте?
20. Як будувати графіки за допомогою бібліотеки matplotlib?

2.5 Зміст звіту

Звіт має складатися з таких розділів:

1. Тема і мета роботи;
2. Завдання до роботи;
3. Результати виконання завдань;
4. Відповіді на контрольні запитання;
5. Висновки, що відображають отримані результати роботи та їх критичний аналіз.

3 ЛАБОРАТОРНА РОБОТА №3 ЗАДАЧА КЛАСИФІКАЦІЇ

3.1 Мета роботи

Ознайомитися та отримати навички роботи з програмою WEKA та бібліотеками мови програмування Python для вирішення задачі класифікації. На практиці вивчити роботу алгоритмів класифікації, навчитися інтерпретувати результати їх роботи та вибирати найкращий метод для вирішення поставленого завдання.

3.2 Основні теоретичні відомості

Класифікація – це віднесення об’єкта до однієї з вже відомих категорій на підставі його ознак (атрибутів).

Задача класифікації – це задача розбиття множини об’єктів або спостережень на апріорно задані групи, так звані класи, всередині кожної з яких вони передбачаються схожими один на одного, мають приблизно однакові властивості і ознаки. При цьому рішення отримується на основі аналізу значень атрибутів.

Класифікація є однією з найважливіших задач інтелектуального аналізу даних. Вона застосовується в маркетингу при оцінці кредитоспроможності позичальників, визначенні лояльності клієнтів, розпізнаванні образів, медичній діагностиці та багатьох інших додатках. Якщо аналітику відомі властивості об’єктів кожного класу, то якщо нове спостереження належить до певного класу, дані властивості автоматично поширюються і на нього.

Класифікація відноситься до стратегії навчання з учителем (Supervised Learning), яке також називають контрольованим або керованим навчанням.

Завданням класифікації часто називають пророкуванням категоріальної залежної змінної (тобто залежної змінної, яка є категорією) на основі вибірки безперервних і/або категоріальних змінних. Наприклад, можна передбачити, який саме товар захоче купити клієнт, хто з клієнтів фірми є потенційним покупцем певного товару, а хто — ні; хто скористається послугою фірми, а хто — ні тощо.

Зазвичай процес вирішення задачі класифікації складається з таких кроків: завантаження даних, попередня обробка даних,

формування вибірок для тренування та тестування, тренування та перевірка якості класифікатора (моделі класифікації).

Серед популярних алгоритмів класифікації можна виділити такі:

- 0R чи Zero Rule класифікація, тобто прогнозування середнього значення для числового класу та моди для категоріального;
- класифікація за одним правилом 1R чи One Rule;
- наївна Байєсова класифікація;
- методи побудови дерев рішень;
- метод опорних векторів;
- метод k найближчих сусідів kNN.

3.2.1 Метрики якості класифікаторів

Матриця помилок – це спосіб розбити об'єкти на чотири категорії в залежності від комбінації істинної відповіді та відповіді алгоритму.

Матриця помилок являє собою матрицю розміром $L \times L$, де L – число класів, ij -й елемент матриці (i – рядок, j – стовпець) дорівнює числу об'єктів з i -го класу, які були віднесені до j -го. Число вірно класифікованих об'єктів дорівнює сумі елементів, що стоять на головній діагоналі.

Результати добре навченого класифікатора покажуть матрицю помилок, в якій найбільші значення стоять на діагоналі матриці, а невеликі значення (в ідеалі нулі) – на інших позиціях.

Розглянемо матрицю помилок побудовану для двох класів «так» та «ні» (табл. 2.1).

Таблиця 2.1 – Матриця помилок для двох класів

		Передбачений клас		
		Так	Ні	
Реальний клас	Так	Істинно позитивні (TP)	Хибно негативні (FN)	$P = TP + FN$
	Ні	Хибно позитивні (FP)	Істинно негативні (TN)	$N = FP + TN$
		$P' = TP + FP$	$N' = FN + TN$	

На діагоналі матриці знаходяться істинно позитивні (true positive, TP) і істинно негативні (true negative, TN) екземпляри. Примірники, які відносяться до класу «так», але були віднесені класифікатором до класу «ні» називаються хибно негативними (false negative, FN). Примірники,

які відносяться до класу «ні», але були віднесені класифікатором до класу «так» називаються хибно позитивними (false positive, FP).

Інтуїтивно зрозумілою, очевидною, але майже невикористовуваною метрикою є accuracy (success rate, recognition rate) – частка правильних відповідей алгоритму та misclassification (error rate) – частка помилкових відповідей алгоритму:

$$accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

$$misclassification = 1 - accuracy = \frac{FP + FN}{TP + TN + FP + FN}$$

Важливо правильно оцінювати втрати і вигоди (costs and benefits), пов'язані з класифікаційною моделлю. Втрати, пов'язані з хибно позитивними передбаченнями (як наприклад, невірне передбачення, що хворий пацієнт є здоровим), набагато більші, ніж хибно негативні передбачення (здоровий пацієнт віднесений до хворих). В таких випадках ми можемо віддати перевагу одному типу помилки над іншим шляхом призначення їм різних значень втрат, пов'язаних з перейменуванням класів.

Наприклад, розглянемо задачі видачі кредиту банком. Втрати при видачі кредиту неплатнику набагато вище, ніж втрати від не видачі кредиту особі, яка зуміла би виплатити свій борг.

Розглянемо тепер проблему нерівномірного розподілу класів, коли важливий для задачі клас є рідкісним в вибірці. Це означає, що розподіл набору даних відображає значну більшість негативних примірників і меншість позитивних примірників.

Наприклад, у задачі розпізнавання шахрайства цікавим для нас класом (позитивним) є клас «шахрайство», який з'являється набагато рідше, ніж негативний клас «не шахрайство». У медичній задачі до такого рідкісного класу може бути віднесений клас «злаякісна пухлина».

Припустимо, що ми навчили класифікатор класифікувати медичний набір даних, в якому цільовим атрибутом є атрибут «злаякісна пухлина», який може приймати значення «так» і «ні». Параметр точності ассигасу рівний 97% може показати, що класифікатор досить точний. Однак якщо у всій вибірці було тільки 3%

примірників, що відносяться до злоякісних пухлин? Ясно, що в такому випадку точність розпізнавання в 97% не може бути прийнятною. У цьому випадку класифікатор може правильно розпізнавати більшість негативних екземплярів (не злоякісна) і помилково класифікувати всі позитивні примірники (злоякісна).

Отже метрики accuracy та misclassification марні для задач з нерівними класами. Таким чином нам необхідні інші параметри, що дозволяють оцінити наскільки добре класифікатор розпізнає позитивні примірники і негативні.

Параметр «precision» (точність) розраховується як:

$$precision = \frac{TP}{TP + FP} = \frac{TP}{P'}$$

Його значення дорівнює відсотку вірно класифікованих об'єктів із всіх об'єктів, віднесених алгоритмом до класу, що розглядається (відношення діагонального елемента до суми елементів стовпця).

Параметр «recall» (TP rate, sensitivity) розраховується таким чином:

$$recall = \frac{TP}{TP + FN} = \frac{TP}{P}$$

Для класу, що розглядається, значення цього параметру дорівнює відсотку вірно класифікованих об'єктів класу (отримується діленням діагонального елемента матриці помилок на суму елементів в його рядку). Іншими словами, параметр чутливості показує долю позитивних екземплярів, які були вірно розпізнані.

Параметр «FP rate» розраховується за формулою:

$$FP\ rate = \frac{FP}{FP + TN} = \frac{FP}{N}$$

Його значення дорівнює відсотку об'єктів інших класів, які помилково були занесені в клас, що розглядається (якщо з матриці викреслити рядок класу, що розглядається, то значення дорівнюватиме сумі елементів стовпця цього класу, поділений на суму всіх елементів).

Параметр «TN rate» (специфічність, specificity) дорівнює:

$$TN\ rate = \frac{TN}{TN + FP} = \frac{TN}{N}$$

Він показує частину негативних екземплярів, які були вірно розпізнані.

Розглянемо такий приклад навчання класифікатора.

Таблиця 2.2 – Приклад навчання класифікатора

Клас	Так	Ні	Всього	Розпізнавання (%)
Так	90	210	300	90 / 300 = 30,00 (sensitivity)
Ні	140	9560	9700	9560 / 9700 = 98,56 (specificity)
Всього	230	9770	10000	9650 / 10000 = 96,50 (accuracy)

Можна помітити, що хоча класифікатор показав загальну високу точність класифікації, його можливості правильно розпізнавати позитивний (рідкісний) клас дуже низькі (що видно з величини параметра чутливості). У той же час параметр специфічності високий, що означає, що класифікатор може точно розпізнавати негативні класи.

Саме введення precision не дозволяє нам записувати всі об'єкти в один клас, так як в цьому випадку ми отримуємо зростання рівня False Positive.

Precision і recall не залежить, на відміну від accuracy та error rate, від співвідношення класів і тому застосовні в умовах незбалансованих вибірок.

Параметри precision і recall також широко використовуються в класифікації. Recall демонструє здатність алгоритму виявляти даний клас взагалі, а precision – здатність відрізнати цей клас від інших класів.

Ідеальне значення параметра recall в 1,0 для класу означає, що кожен екземпляр був віднесений класифікатором до його класу, але це нічого не говорить нам про те, скільки інших екземплярів були неправильно класифіковані та віднесені до іншого класу.

Існує тенденція зворотного взаємозв'язку між параметрами precision і recall, тобто існує можливість збільшити один параметр за рахунок зменшення іншого. Приміром, наш медичний класифікатор може досягти високого значення параметра precision відносячи всі екземпляри класу злоякісних пухлин до класу злоякісних, але при цьому може мати низьке значення параметра recall відносячи до класу злоякісних також негативні екземпляри. Зазвичай ці два параметри

розглядаються сумісно, а в реальній практиці стоїть завдання знайти оптимальний баланс між цими двома метриками.

Альтернативний шлях застосування цих параметрів – це їх об'єднання в одному параметрі F-measure (F1 score або F-score):

$$F - measure = 2 * \frac{precision * recall}{precision + recall}$$

Параметр «F-measure» – це середнє гармонійне між precision та recall. Цей параметр досягає максимуму при значеннях, рівних одиниці, і близький до нуля, якщо один з аргументів близький до нуля.

Одним із способів оцінити модель в цілому, не прив'язуючись до конкретного порогу, є AUC-ROC (або ROC AUC) – площа (Area Under Curve) під кривою помилок (Receiver Operating Characteristic curve). Дана крива представляє із себе лінію від (0,0) до (1,1) в координатах True Positive Rate (TPR) і False Positive Rate (FPR):

$$TPR = \frac{TP}{TP + FN} = recall$$

$$FPR = \frac{FP}{FP + TN} = FP\ rate$$

В ідеальному випадку, коли класифікатор не робить помилок (FPR = 0, TPR = 1) ми отримаємо площу під кривою, що дорівнює одиниці; в іншому випадку, коли класифікатор випадково видає ймовірності класів, AUC-ROC буде прагнути до 0.5, так як класифікатор видаватиме однакову кількість TP і FP.

Кожна точка на графіку відповідає вибору деякого порога. Площа під кривою в даному випадку показує якість алгоритму (більше – краще), крім цього, важливою є крутизна самої кривої – ми хочемо максимізувати TPR, мінімізуючи FPR, а отже, наша крива в ідеалі повинна прагнути до точки (0,1).

Precision і recall також використовують для побудови кривої та аналогічно AUC-ROC знаходять площу під нею – AUC-PR. Варто відзначити, що на маленьких наборах даних площа AUC-PR може бути занадто оптимістичною, адже обчислюється за методом трапецій, але зазвичай в таких завданнях даних достатньо.

Розглянемо ще один параметр оцінки точності класифікації. В табл. 2.3 представлений приклад матриці помилок задачі класифікації з трьома класами.

Таблиця 2.3 – Приклад №1 матриці помилок

		Передбачений клас			
		А	В	С	Всього
Реальний клас	А	88	10	2	100
	В	14	40	6	60
	С	18	10	12	40
	Всього	120	60	20	

В цьому прикладі тестова вибірка містить 200 примірників (сума дев'яти елементів матриці). Класифікатор для тестової вибірки передбачив 120 екземплярів класу А, 60 – класу В, 20 – класу С, та при цьому $88 + 40 + 12 = 140$ з них правильно класифіковані. Відсоток правильно класифікованих об'єктів для цього прикладу дорівнює 70%.

Що якби на цій самій вибірці працював би випадковий класифікатор, який передбачив би таку ж кількість примірників кожного класу. Розглянемо таблицю 2.4.

Таблиця 2.4 – Приклад №2 матриці помилок

		Передбачений клас			
		А	В	С	Всього
Реальний клас	А	60	30	10	100
	В	36	18	6	60
	С	24	12	4	40
	Всього	120	60	20	

В першому рядку 100 екземплярів класу А розподілені вони в такій же пропорції (120:60:20). Точно так же розподілені примірники другого й третього рядка. Загальні значення в рядках та стовпцях не змінилися, а змінилися значення матриці. Таким чином, випадковий класифікатор дає $60 + 18 + 4 = 82$ правильно класифікованих примірників.

Параметр Каппа розраховує це очікуване значення (виводячи його з успішності класифікатора) та виражає результат у вигляді відношення: у чисельнику $140 - 82 = 58$ примірників поліпшення в порівнянні з випадковим прогнозуванням, а в знаменнику – все можливе поліпшення $200 - 82 = 118$. Для наведеного вище прикладу

параметр Каппа дорівнює 49,2%. Максимальне значення параметра Каппа 100% для ідеального передбачення, а мінімальне 0% – для випадкового.

Загалом, можна сказати, що статистичний параметр Каппа використовується для оцінки згоди між прогнозованою і спостережуваною категоризацією набору даних з поправкою на випадковість.

3.2.2 Критерії порівняння роботи класифікаторів

На додаток до параметрів, заснованих на точності, класифікатори можуть бути порівняні за такими параметрами:

- швидкість роботи – іншими словами обчислювальні витрати, пов'язані з навчанням і застосуванням даного класифікатора.
- стійкість до помилок, робастність – можливість класифікатора робити правильні передбачення на зашумлених даних або даних з пропущеними значеннями. Даний параметр зазвичай оцінюється за допомогою синтетичних наборів даних із внесеними шумами і втраченими значеннями атрибутів.
- масштабованість – можливість будувати ефективний класифікатор на великих вибірках. Даний параметр оцінюється за допомогою наборів даних, що збільшуються.
- інтерпретованість – рівень розуміння та можливості проникнути в суть даних, які надає класифікатор. Інтерпретованість є суб'єктивним параметром і тому його важко оцінити. Дерева рішень та класифікаційні правила можуть бути легко інтерпретовані, проте їх інтерпретованість зменшується з їх ускладненням.

3.2.3 Робота з програмою WEKA

Для вирішення задачі класифікації за допомогою програми WEKA можна використати функції, що надає графічний інтерфейс користувача середовища Explorer, а саме: вкладку Classify (класифікація).

Після завантаження файлу з даними необхідно вибрати алгоритм, встановити необхідні налаштування та побудувати класифікатор. В результаті буде отримано класифікатор та основну статистичну інформацію про якість його роботи.

Більш детальний опис роботи з вкладкою Classify графічного інтерфейсу користувача програми WEKA наведено в додатку А.

Програма WEKA надає можливість побудови класифікатора за такими алгоритмами (у дужках наведено назву в програмі WEKA, при цьому перше слово перед крапкою означає тип алгоритму класифікації і також вказує назву папки, в якій знаходиться метод):

- 0R чи Zero Rule класифікація, тобто прогнозування середнього значення для числового класу та моди для категоріального (rules.ZeroR);

- класифікація за одним правилом 1R чи One Rule (rules.OneR);

- покриваючий метод PRISM (rules.Prism);

- наївна Байєсова класифікація (bayes.NaiveBayes);

- методи побудови дерев рішень CART (trees.simpleCart) та C4.5 (trees.J48); – метод опорних векторів (functions.SMO);

- метод k найближчих сусідів kNN (lazy.IBk).

Для додавання найпростіших методів в WEKA необхідно встановити пакети simpleEducationalLearningSchemes та simpleCART.

Розглянемо параметри налаштування алгоритмів, що використовуються в роботі (табл. 2.5). Додаткову інформацію про алгоритми, їх параметри та вимоги до оброблюваних даних можна отримати у вікні налаштувань алгоритмів на панелі «About» в програмі WEKA

Для алгоритмів Prism, Id3, ZeroR параметрів, що настроюються, немає.

Таблиця 2.5 – Параметри налаштування класифікаторів

Метод	Параметр
OneR	MinBucketSize – використовується для дискретизації числових атрибутів.
NaiveBayes	displayModelInOldFormat – відображення побудованої моделі у старому форматі, що підходить, коли атрибут класу приймає багато значень. Новий формат краще у випадку, коли менше класів і багато атрибутів. useKernelEstimator – для оцінки числових атрибутів використовувати оціночну функцію відмінну від нормального розподілу. useSupervisedDiscretization – використовувати дискретизацію з учителем для перетворення числових атрибутів у номінальні.

Продовження табл. 2.5

Метод	Параметр
J48	binarySplits – використовувати бінарний поділ на категоріальних атрибутах для побудови дерев. confidenceFactor – довірчий рівень, використовується для відсікання гілок (малі значення – сильніше відсікання). minNumObj – мінімальна кількість примірників у листі. reducedErrorPruning – який алгоритм відсікання гілок використовувати. saveInstanceData – чи зберігати навчальну інформацію для візуалізації. subtreeRaising – чи використовувати операцію підняття піддерев при відсіканні гілок. unpruned – чи залишити дерево повним. seLaplace – використовувати оціночну функцію Лапласа для підрахунку ймовірностей в листках.
SMO	buildLogisticModels – чи застосовувати логістичні моделі до виходів (для належної оцінки ймовірностей). c – параметр складності C. kernel – функція ядра. epsilon – параметр точності (не змінювати). filterType – чи буде змінена початкова інформація і яким чином (нормалізація або стандартизація). toleranceParameter – допустиме відхилення (не змінювати).
IBk	KNN – кількість сусідів. crossValidate – чи буде використовуватися для вибору оптимальної кількості сусідів крос-перевірка hold-one-out. distanceWeighting – метод вибору вагових коефіцієнтів для відстаней. meanSquared – чи використовується середньоквадратична помилка, чи середня абсолютна помилка для кросперевірки під час вирішення завдання регресії. nearestNeighbourSearchAlgorithm – алгоритм пошуку найближчих сусідів. windowSize – максимальна кількість примірників, дозволених в навчальному пулі. Додавання додаткових примірників понад цього значення призведе до видалення старих екземплярів. Значення 0 означає відсутність межі.

Розглянемо результати роботи класифікаторів в WEKA (Classifier output).

Секція «Run information» містить таку інформацію:

- метод класифікації (scheme);
- назва набору даних, на якому проводилося навчання (relation);
- кількість примірників у вихідній вибірці (instances);
- атрибути, що характеризують об'єкти вибірки (attributes);
- відомості про тестову вибірку (test mode).

Секція «Classifier model» містить параметри налаштованого класифікатора і час, затрачений для побудови моделі. Залежно від типу класифікатора дана область буде містити різну інформацію:

- для алгоритмів, що будують правила, будуть відображені отримані правила;

- для байєсівських класифікаторів будуть перераховані розраховані ймовірності для всіх можливих комбінацій атрибутзначення-клас;

- для класифікаторів, заснованих на побудові дерев, відображається текстове представлення отриманого дерева; в дужках навпроти кожного листа вказана кількість примірників, які до нього віднесені; якщо в лист потрапляють екземпляри декількох класів, через слеш буде вказана кількість примірників, які відносяться до домішок;

- для функціональних методів виводяться значення коефіцієнтів побудованої функціональної моделі;

- для методу k найближчих сусідів відображаються налаштування класифікатора.

Секція «Predictions» буде відображена, якщо в налаштуваннях тестування класифікатора обрана опція «Output predictions». У ній для всіх примірників тестової вибірки будуть відображені результати класифікації, отримані за допомогою навченого класифікатора.

Секція оцінки побудованої моделі «Evaluation» містить кілька підпунктів.

«Summary» містить загальну статистику роботи класифікатора:

- кількість та відсоток правильно і неправильно класифікованих примірників (Correctly and Incorrectly Classified Instances), загальна кількість примірників (Total Number of Instances);

- параметр Каппа (Kappa statistic);

- статистичні параметри помилки класифікації (Mean absolute error, Root mean squared error, Relative absolute error, Root relative squared error).

«Detailed Accuracy By Class» містить такі параметри точності

класифікації по кожному з класів: precision, recall TP Rate, FP Rate, F-Measure, AUC-ROC.

«Confusion Matrix» містить матрицю помилок.

3.2.4 Робота з Python

Для вирішення задачі класифікації за допомогою мови програмування Python можна використовувати пакет scikit-learn – один з найпопулярніших інструментів для машинного навчання та інтелектуального аналізу даних.

Scikit-learn (раніше scikits.learn, а також відомий як sklearn) – це безкоштовна бібліотека машинного навчання для мови програмування Python. Вона містить різні алгоритми класифікації, регресії та кластеризації, включаючи ті, що підтримують векторні машини, випадкові ліси, збільшення градієнта, k-засоби та DBSCAN. Пакет призначений для взаємодії з числовими та науковими бібліотеками Python NumPy та SciPy.

Для встановлення пакета scikit-learn необхідно виконати таку команду:

```
pip install scikit-learn
```

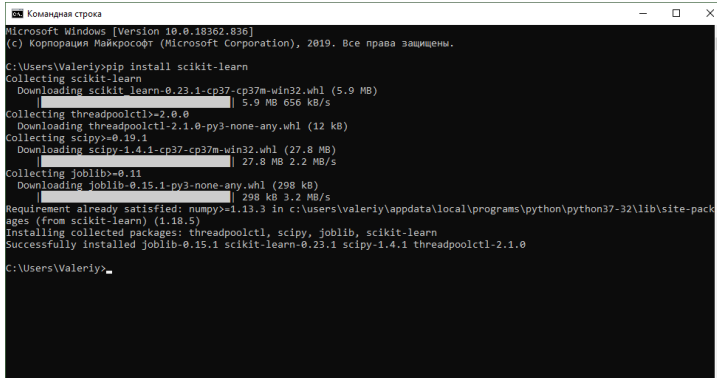
В результаті виконання цієї команди (рис. 3.1) буде встановлено пакет scikit-learn та бібліотеки від яких він залежить :

- scikit-learn;

- scipy – відкрита бібліотека високоякісних наукових інструментів для мови програмування Python. SciPy містить модулі для оптимізації, інтегрування, спеціальних функцій, обробки сигналів, обробки зображень, генетичних алгоритмів, розв’язування звичайних диференціальних рівнянь та інших задач, які розв’язуються в науці і при інженерній розробці. Бібліотека розробляється для тієї ж аудиторії, що і MATLAB та Scilab. Для візуалізації при використанні SciPy часто застосовують бібліотеку Matplotlib, яка є аналогом засобів виводу графіки MATLAB;

- joblib – набір інструментів для забезпечення легкого конвеєру в Python;

- threadpoolctl – помічник Python для обмеження кількості потоків, що використовуються у резервній системі загальних нативних бібліотек, що використовуються для наукових обчислень та обробки даних.



```

Командная строка
Microsoft Windows [Version 10.0.18362.836]
(c) Корпорация Майкрософт (Microsoft Corporation), 2019. Все права защищены.

C:\Users\Valeriy>pip install scikit-learn
Collecting scikit-learn
  Downloading scikit_learn-0.23.1-cp37-cp37m-win32.whl (5.9 MB)
    |#####| 5.9 MB 656 kB/s
Collecting threadpoolctl>=2.0.0
  Downloading threadpoolctl-2.1.0-py3-none-any.whl (12 kB)
Collecting scipy>=0.19.1
  Downloading scipy-1.4.1-cp37-cp37m-win32.whl (27.8 MB)
    |#####| 27.8 MB 2.2 MB/s
Collecting joblib>=0.11
  Downloading joblib-0.15.1-py3-none-any.whl (298 kB)
    |#####| 298 kB 3.2 MB/s
Requirement already satisfied: numpy>=1.13.3 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-pack
ages (from scikit-learn) (1.18.5)
Installing collected packages: threadpoolctl, scipy, joblib, scikit-learn
Successfully installed joblib-0.15.1 scikit-learn-0.23.1 scipy-1.4.1 threadpoolctl-2.1.0

C:\Users\Valeriy>

```

Рисунок 3.1 – Встановлення пакета scikit-learn

Після виконання попередньої обробки даних можна приступити до побудови моделей класифікації та їх оцінювання.

Зазвичай перед використанням якогось пакета ми повністю імпортуємо його. Імпортувати пакет scikit-learn можна за допомогою такого коду:

```
import sklearn
```

Проте в даному випадку так робити не варто. Пакет scikit-learn складається з великої кількості різних функцій та алгоритмів, які в повній мірі не використовуються але займають певний простір та уповільнюють швидкість роботи програми. Тому доцільно імпортувати тільки ті модулі, які буде задіяно в програмному коді.

В якості вибірки з даними візьмемо вже попередньо оброблений набір даних:

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv("D:\\iris.csv")
```

```
print(data.info())
```

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   sepal.length    150 non-null   float64
1   sepal.width     150 non-null   float64
2   petal.length    150 non-null   float64

```

```

3   petalwidth    150 non-null    float64
4   class         150 non-null    object
dtypes: float64(4), object(1)
memory usage: 5.3+ KB

```

```
print(data.head(5))
```

```

      sepallength  sepalwidth  petallength  petalwidth      class
0           5.1         3.5         1.4         0.2  Iris-setosa
1           4.9         3.0         1.4         0.2  Iris-setosa
2           4.7         3.2         1.3         0.2  Iris-setosa
3           4.6         3.1         1.5         0.2  Iris-setosa
4           5.0         3.6         1.4         0.2  Iris-setosa

```

Перед початком навчання моделі класифікації необхідно відокремити цільовий клас від атрибутів та розділити набір даних на дві вибірки: для навчання та для тестування.

Відокремимо цільовий атрибут (клас) від інших атрибутів за допомогою можливостей структур даних pandas.

```

Y = data["class"]
X = data.drop(["class"], axis = 1)

```

Бібліотека scikit-learn надає багато класів з алгоритмами для розподілу даних у вибірки для навчання та тестування. Такі класи можна знайти в модулі sklearn.model_selection. Для перехресної перевірки можна використати такі: GroupKFold, KFold, LeaveOneOut, RepeatedKFold, StratifiedKFold та інші.

Перехресна перевірка (cross-validation) – це метод, при якому для навчання використовується весь набір даних, проте навчання відбувається багаторазово, і в якості валідаційної вибірки на кожному кроці виступають різні частини датасета. Підсумковий результат являє собою усереднення отриманих результатів.

Приклад роботи:

```
from sklearn.model_selection import KFold
```

```

# Створення екземпляра класа
kf = KFold(n_splits = 5)

```

```

# Перебір заданої кількості наборів для перехресної перевірки
for train_index, test_index in kf.split(X):
    X_train, X_test = X.loc[train_index], X.loc[test_index]
    Y_train, Y_test = Y[train_index], Y[test_index]
    # Щось робимо з набором даних (побудова класифікатора,
    # аналіз роботи класифікатора, тощо)

```

Параметр `n_splits` вказує на те, скільки розподілів необхідно зробити.

Більш простим підходом є звичайний розподіл вибірки на дві частини. Для отримання вибірок для навчання та тестування використаємо функцію `train_test_split()`:

```
from sklearn.model_selection import train_test_split

X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
test_size = 0.33, random_state = 10)
```

Параметр `test_size` визначає пропорцію, в якій має бути розділено вибірку (частка елементів, які потраплять в вибірку для тестування), а `random_state` – сім для генератора випадкових чисел.

Бібліотека `scikit-learn` надає багато класів з алгоритмами для побудови, навчання та тестування моделей класифікатора.

Розберемо роботу з `KNeighborsClassifier`. `KNeighborsClassifier` – класифікатор що працює за методом k-найближчих сусідів. Для класифікації об'єктів у рамках простору властивостей використовуються відстані (зазвичай евклідові), порашовані до усіх інших об'єктів. Вибираються об'єкти, до яких відстань найменша, а об'єкт присвоюється тому класу, який є найбільш поширеним серед сусідів.

Перш за все необхідно імпортувати клас, який відповідає за використовуваний метод:

```
from sklearn.neighbors import KNeighborsClassifier
```

Далі створюємо екземпляр цього класу й задаємо необхідні параметри (для `KNeighborsClassifier` це `n_neighbors` – кількість сусідів, які беруться до уваги):

```
KNN = KNeighborsClassifier(n_neighbors = 3)
```

Для навчання класифікатора можна використати функцію `fit()`:

```
KNN.fit(X_train, Y_train)
```

В результаті навчання отримано класифікатор, який можна використати для прогнозування класа екземпляра вибірки. Класифікатори можуть робити прогнози й зберігати результат в будь-яку змінну за допомогою функції `predict()`:

```
Y_predict = KNN.predict(X_test)
```

Робота з будь-яким іншим методом класифікації складається з аналогічних кроків, проте перед використанням інших моделей варто


```

| | | |--- petalwidth <= 1.55
| | | |--- class: Iris-virginica
| | | |--- petalwidth > 1.55
| | | |--- class: Iris-versicolor
| | |--- petalwidth > 1.75
| | |--- class: Iris-virginica

```

Тепер прийшов час оцінити точності класифікатора. Існує декілька способів це зробити.

Модуль `sklearn.metrics` містить різні метрики для оцінки якості побудованих класифікаторів.

За допомогою функції `accuracy_score()` можна отримати частку правильних відповідей алгоритму – `accuracy`:

```

from sklearn.metrics import accuracy_score
print(accuracy_score(Y_test, Y_predict))

```

0.96

За допомогою функції `confusion_matrix()` можна отримати матрицю помилок:

```

from sklearn.metrics import confusion_matrix
print(confusion_matrix(Y_test, Y_predict))

```

```

[[15  0  0]
 [ 0 17  2]
 [ 0  0 16]]

```

За допомогою функції `classification_report()` можна отримати значення `precision`, `recall`, `F-measure`, `support` та інші.

```

from sklearn.metrics import classification_report
print(classification_report(Y_test, Y_predict))

```

	precision	recall	f1-score	support
Iris-setosa	1.00	1.00	1.00	15
Iris-versicolor	1.00	0.89	0.94	19
Iris-virginica	0.89	1.00	0.94	16
accuracy			0.96	50
macro avg	0.96	0.96	0.96	50
weighted avg	0.96	0.96	0.96	50

Приклад роботи з декількома класифікаторами (навчання та прогнозування) з подальшою візуалізацією результатів роботи алгоритмів:

```

# Імпорт необхідних модулів бібліотеки
from sklearn.dummy import DummyClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

# Оголошення словника, який зберігає назву та
# модель класифікації
classifiers = {
    "Zero rule" : DummyClassifier(strategy="most_frequent"),
    "K-nearest neighbors" : KNeighborsClassifier(n_neighbors
= 5),
    "Decision tree" : DecisionTreeClassifier(max_depth = 5),
    "Gaussian naive Bayes" : GaussianNB(),
    "Linear SVM" : SVC(kernel="linear", C=0.025)}

# Словник, який має зберігати результати класифікації
classifiers_results = {}

# Перебір всіх ключей словника (назв моделей)
for key in classifiers.keys():
# Навчання моделі класифікації на навчальній вибірці
    classifiers[key].fit(X_train, Y_train)
# Прогнозування результатів для тестової вибірки
    Y_predict = classifiers[key].predict(X_test)
# Запис точності класифікації до результатів
    classifiers_results[key] = accuracy_score(Y_test,
Y_predict)

# Запис результатів до серії та демонстрація
results = pd.Series(classifiers_results)
print(results)

Zero rule                0.30
K-nearest neighbors      0.96
Decision tree            0.90
Gaussian naive Bayes     0.94
Linear SVM               0.98
dtype: float64

# Побудова та відображення графіка, що демонструє точності
# передбачення різних класифікаторів
results.plot(kind = "bar")
plt.show()

```

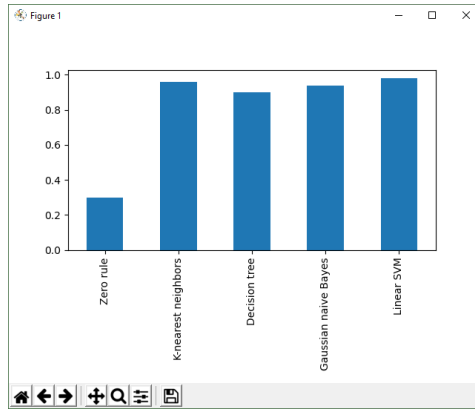


Рисунок 3.3 – Порівняння точності алгоритмів класифікації

3.3 Завдання на лабораторну роботу

1. Вирішити задачу класифікації за допомогою програми WEKA:
 - завантажити попередньо оброблену вибірку з лабораторної роботи № 2 (ARFF файл);
 - побудувати та оцінити моделі класифікації за допомогою всіх розглянутих в лабораторній роботі алгоритмів в WEKA;
 - змінюючи параметри налаштування алгоритмів, досягти найліпшої якості навчання різних класифікаторів;
 - порівняти отримані моделі класифікації за допомогою модуля Experimenter.
2. Вирішити задачу класифікації за допомогою засобів мови програмування Python:
 - завантажити попередньо оброблену вибірку з лабораторної роботи № 2 (CSV файл);
 - побудувати та оцінити моделі класифікації за допомогою всіх розглянутих в лабораторній роботі алгоритмів бібліотеки scikit-learn;
 - змінюючи параметри налаштування деяких алгоритмів, досягти найліпшої якості навчання класифікаторів;
 - порівняти отримані моделі класифікації за допомогою графічних засобів бібліотеки matplotlib.
3. Відповісти на 5 контрольних запитань.
4. Оформити звіт з роботи.

3.4 Контрольні питання

1. Що таке задача класифікації? Наведіть практичний приклад.
2. Що таке навчання з учителем і без учителя? До якої стратегії відноситься задача класифікації?
3. Які алгоритми класифікації ви знаєте?
4. Опишіть один із методів класифікації.
5. Навіщо потрібні дві вибірки: навчальна і тестова? Які існують підходи для поділу вихідної вибірки на навчальну і тестову?
6. Які метрики якості класифікаторів ви знаєте?
7. Що таке матриця помилок? Як її інтерпретувати?
8. Що означають параметри accuracy, precision, recall (sensitivity), specificity, F-measure? Як їх розрахувати?
9. Опишіть параметри AUC-ROC та AUC-PR.
10. Що таке параметр Каппа? Що він показує?
11. Що таке аналіз втрати-виграші? Навіщо він?
12. Опишіть критерії порівняння роботи класифікаторів.
13. Як навчити й оцінити модель класифікації в програмі WEKA?
14. Як порівняти роботу декількох класифікаторів в WEKA?
15. Для чого використовується пакет scikit-learn?
16. Як розділити вибірку на навчальну та тестову за допомогою бібліотеки scikit-learn?
17. Опишіть принципи роботи з моделями класифікації за допомогою бібліотеки scikit-learn?
18. Які метрики якості класифікаторів представлено в бібліотеці scikit-learn?
19. Як порівняти роботу декількох класифікаторів за допомогою Python?

3.5 Зміст звіту

Звіт має складатися з таких розділів:

1. Тема і мета роботи;
2. Завдання до роботи;
3. Результати виконання завдань;
4. Відповіді на контрольні запитання;
5. Висновки, що відображають отримані результати роботи та їх критичний аналіз.

4 ЛАБОРАТОРНА РОБОТА № 4 ПРОГНОЗУВАННЯ, ЗАДАЧА РЕГРЕСІЇ

4.1 Мета роботи

Ознайомитися та отримати навички роботи з програмою WEKA та бібліотеками мови програмування Python для прогнозування та вирішення задачі регресії. На практиці вивчити роботу алгоритмів, що вирішують задачу регресії, навчитися інтерпретувати результати їх роботи та вибирати найкращий метод для вирішення поставленого завдання.

4.2 Основні теоретичні відомості

Регресія – це встановлення залежності безперервних вихідних значень від вхідних параметрів.

Задача регресії (регресійний аналіз) – це метод моделювання вимірюваних даних та дослідження їх властивостей. Дані складаються з пар значень залежної змінної (змінної відгуку) та незалежних параметрів (що пояснюють змінну). Регресійний аналіз є розділом математичної статистики та машинного навчання. Передбачається, що залежна змінна являє собою суму значень деякої моделі з випадковою величиною. Щодо характеру розподілу цієї величини робляться припущення, звані гіпотезою породження даних. Для підтвердження або спростування цієї гіпотези виконуються статистичні тести, що називають аналізом залишків. При цьому передбачається, що незалежна змінна не містить помилок. Регресійний аналіз використовується для прогнозу, аналізу часових рядів, тестування гіпотез та виявлення прихованих взаємозв'язків в даних.

Побудовану модель називають регресійною. Регресійна модель – це функція незалежної змінної від параметрів з доданою випадковою величиною. Параметри моделі налаштовуються таким чином, що модель якнайкраще наближує розрахований результат до реальних даних. Критерієм якості наближення (цільовою функцією) зазвичай є середньоквадратична помилка: сума квадратів різниці значень моделі й залежної змінної для всіх значень незалежної змінної як аргумента.

Регресія відноситься до стратегії навчання з учителем (Supervised Learning), проте на відміну від класифікації, де множина допустимих

відповідей скінченна, в регресії допустимою відповіддю є дійсне число або числовий вектор.

Зазвичай процес вирішення задачі регресії складається з таких кроків: завантаження даних, попередня обробка даних, формування вибірок для тренування та тестування, побудова та перевірка якості регресійної моделі.

Серед популярних алгоритмів регресійного аналізу можна виділити такі:

- лінійна регресія;
- регресійні дерева й модельні дерева;
- адаптовані методи опорних векторів;
- метод k найближчих сусідів kNN.

4.2.1 Метрики якості регресійних моделей

Наведені параметри для оцінки якості класифікаторів більш корисні для опису задач класифікації ніж для завдань регресії. Для задачі регресії помилки прогнозування не просто присутні або відсутні, а мають різні числові значенні. Для оцінки якості числових прогнозів можуть бути використані альтернативні параметри, деякі з яких наведено в табл. 4.1.

Таблиця 4.1 – Метрики якості вирішення задачі регресії

Параметр	Формула для розрахунку
Середній квадрат помилки (Mean-squared error)	$\frac{(p_1 - a_1)^2 + \dots + (p_n - a_n)^2}{n}$
Середньоквадратична помилка (Root mean-squared error)	$\sqrt{\frac{(p_1 - a_1)^2 + \dots + (p_n - a_n)^2}{n}}$
Середня абсолютна помилка (Mean-absolute error)	$\frac{ p_1 - a_1 + \dots + p_n - a_n }{n}$
Відносний квадрат помилки (Relative-squared error)*	$\frac{(p_1 - a_1)^2 + \dots + (p_n - a_n)^2}{(a_1 - \bar{a})^2 + \dots + (a_n - \bar{a})^2}$
Відносна середньоквадратична помилка (Root relative-squared error)*	$\sqrt{\frac{(p_1 - a_1)^2 + \dots + (p_n - a_n)^2}{(a_1 - \bar{a})^2 + \dots + (a_n - \bar{a})^2}}$

Продовження табл. 4.1

Параметр	Формула для розрахунку
Відносна абсолютна помилка (Relative absolute error)*	$\frac{ p_1 - a_1 + \dots + p_n - a_n }{ a_1 - \bar{a} + \dots + p_n - \bar{a} }$
Коефіцієнт кореляції (Correlation coefficient)**	$\frac{S_{PA}}{\sqrt{S_P * S_A}}$
$S_{PA} = \frac{\sum_i (p_i - \bar{p})(a_i - \bar{a})}{n - 1}, S_P = \frac{\sum_i (p_i - \bar{p})^2}{n - 1}, S_A = \frac{\sum_i (a_i - \bar{a})^2}{n - 1}$	

де $p_1, p_2, \dots, p_n$ – прогнозовані значення для цільового атрибута тестової вибірки;

$a_1, a_2, \dots, a_n$ – реальні значення цільового атрибута;

$\bar{a}$ – середнє арифметичне (* – навчальної, ** – тестової вибірки).

4.2.2 Робота з програмою WEKA

Для вирішення задачі регресії за допомогою програми WEKA можна використати функції, що надає графічний інтерфейс користувача середовища Explorer, а саме: вкладку Classify (класифікація). В цій вкладці наведено не тільки методи для класифікації, а й алгоритми для побудови моделей регресії.

Після завантаження файлу з даними необхідно вибрати алгоритм, встановити необхідні налаштування та побудувати модель регресії. В результаті буде отримано рівняння регресії з відповідними коефіцієнтами та основну статистичну інформацію.

Більш детальний опис роботи з вкладкою Classify графічного інтерфейсу користувача програми WEKA наведено в додатку А.

Програма WEKA надає можливість побудови регресійних моделей за такими алгоритмами (у дужках наведено назву в програмі WEKA, при цьому перше слово перед крапкою означає тип алгоритму, а також вказує назву папки, в якій він знаходиться):

- лінійна регресія (functions.LinearRegression);
- регресійні дерева і модельні дерева (trees.M5P);
- метод опорних векторів, модифікований для вирішення задач регресії (functions.SMOreg);
- метод найближчих сусідів (lazy.IBk).

Розглянемо параметри налаштування використовуваних

алгоритмів у WEKA (табл. 4.2).

Таблиця 4.2 – Параметри налаштування методів

Метод	Параметр
LinearRegression	attributeSelectionMethod – метод відбору атрибутів. eliminateColinearAttributes – виключити колінеарні атрибути. ridge – штраф за великі значення коефіцієнтів регресії (регуляризація Тихонова).
M5P	buildRegressionTree – регресійне або модельне дерево. minNumInstances – мінімальна кількість примірників у листі. saveInstances – зберігати примірники у вузлах для візуалізації. unpruned – будувати дерево без відсікання useUnsmoothed – незгладжене прогнозування.
SMOreg	regOptimizer – алгоритм навчання. Інші параметри можна знайти в таблиці 2.5.
IBk	Основні параметри можна знайти в таблиці 2.5.

4.2.3 Робота з Python

Вирішення задачі регресії за допомогою засобів мови програмування Python аналогічне вирішенню задачі класифікації.

Вирішимо задачу регресії для набору даних з параметрами людини та спрогнозуємо тривалість сну. Спочатку завантажимо вибірку та виконаємо попередню обробку даних:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

data = pd.read_csv("D:\\sleep.csv")

print(data.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 62 entries, 0 to 61
Data columns (total 8 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   body_weight            62 non-null     float64
1   brain_weight           62 non-null     float64
2   max_life_span          58 non-null     float64
3   gestation_time         58 non-null     float64
4   predation_index        62 non-null     int64
5   sleep_exposure_index   62 non-null     int64
6   danger_index           62 non-null     int64
7   total_sleep            58 non-null     float64
dtypes: float64(5), int64(3)
memory usage: 3.9 KB
```

```
print(data.isnull().sum())
```

```
body_weight      0
brain_weight     0
max_life_span    4
gestation_time   4
predation_index  0
sleep_exposure_index  0
danger_index     0
total_sleep      4
dtype: int64
```

Замінемо пропущені дані на середні значення відповідних атрибутів та відокремимо цільовий атрибут за допомогою можливостей структур даних pandas:

```
data = data.fillna(data.mean())
```

```
Y = data["total_sleep"]
X = data.drop(["total_sleep"], axis = 1)
```

Для отримання вибірок для побудови моделі регресії та тестування використаємо функцію `train_test_split()`:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y,
test_size = 0.66, random_state = 10)
```

Бібліотека `scikit-learn` надає багато класів з алгоритмами для побудови та тестування регресійних моделей.

Розберемо роботу з `KNeighborsRegressor`. `KNeighborsRegressor` – клас що реалізує алгоритм побудови регресійних моделей, що працює за адаптованим для задачі регресії класифікаційним методом *k*-найближчих сусідів.

Перш за все необхідно імпортувати клас, який відповідає за використання методу:

```
from sklearn.neighbors import KNeighborsRegressor
```

Далі створюємо екземпляр цього класу й задаємо необхідні параметри:

```
KNN = KNeighborsRegressor()
```

Будуємо (навчаємо) модель регресії за допомогою функції `fit()`:

```
KNN.fit(X_train, Y_train)
```

Спрогнозуємо значення цільової змінної для тестової вибірки за допомогою функції `predict()`:

```
Y_predict = KNN.predict(X_test)
```

Для оцінки якості побудованої моделі регресії використаємо параметр *mean-squared error*, який можна розрахувати за допомогою функції `mean_squared_error()` модуля `sklearn.metrics`.

```
from sklearn.metrics import mean_squared_error
```

```
print(mean_squared_error(Y_test, Y_predict))
```

```
16.581035770424293
```

Змінюючи параметри алгоритма спробуємо досягти найкращої якості моделі регресії. Як метрику для оцінки якості моделі використаємо параметр *mean-absolute error*, який можна розрахувати за допомогою функції `mean_absolute_error()` модуля `sklearn.metrics`. Отримані результати візуалізуємо.

```
# Імпорт функції для розрахунку середньої абсолютної помилки
from sklearn.metrics import mean_absolute_error
```

```
# Словник, який має зберігати оцінку якості моделі
regression_results = {}
```

```
# Перебір значень для параметра n_neighbors
for i in range(1, 11):
```

```
# Оголошення та побудова моделі регресії
```

```
    KNN = KNeighborsRegressor(n_neighbors = i)
```

```

KNN.fit(X_train, Y_train)
# Прогнозування результатів для тестової вибірки
Y_predict = KNN.predict(X_test)
# Запис метрики до результатів
regression_results[i] = mean_absolute_error(Y_test,
Y_predict)

# Запис результатів до серії та демонстрація
results = pd.Series(regression_results)
print(results)

1      3.623549
2      3.465034
3      3.243664
4      3.197981
5      3.217881
6      3.129121
7      3.266905
8      3.277544
9      3.355551
10     3.334790
dtype: float64

# Побудова та відображення графіка, що демонструє середню
# абсолютну помилку для різних значень n_neighbors
results.plot()
plt.show()

```

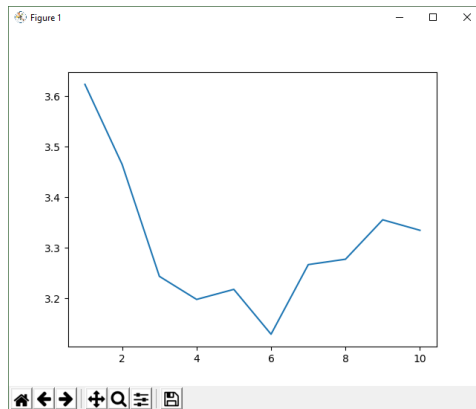


Рисунок 4.1 – Порівняння середньої абсолютної помилки регресійної моделі для різних значень параметра `n_neighbors`

Відповідно до отриманих результатів, можна сказати, що найкращий результат прогнозування досягається при значенні параметра `n_neighbors = 6`.

Побудуємо модель лінійної регресії та оцінимо які атрибути виявилися найбільш важливими для прогнозування:

```
from sklearn.linear_model import LinearRegression

# Оголошення та побудова моделі регресії
LR = LinearRegression()
LR.fit(X_train, Y_train)

# Запис в серію модулів коефіцієнтів атрибутів отриманих
# за допомогою параметра coef_ та демонстрація
coefficients = pd.Series(np.absolute(LR.coef_), index =
X.columns)
print(coefficients)

body_weight          0.015377
brain_weight         0.019024
max_life_span        0.157937
gestation_time       0.020801
predation_index      0.889922
sleep_exposure_index 1.000639
danger_index         2.053726
dtype: float64

coefficients.plot(kind = "barh")
plt.show()
```

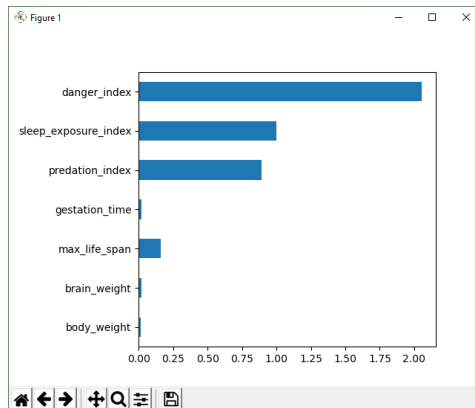


Рисунок 4.2 – Порівняння якості регресійних моделей

Приклад роботи з декількома моделями регресії (побудова та прогнозування) з подальшою візуалізацією результатів роботи алгоритмів:

```
# Імпорт необхідних модулів бібліотеки
from sklearn.linear_model import LinearRegression
from sklearn.neighbors import KNeighborsRegressor
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR

# Оголошення словника, який зберігає назву та модель регресії
models = {
    "Linear regression" : LinearRegression(),
    "K neighbors regressor" : KNeighborsRegressor(n_neighbors
= 6),
    "Decision tree regressor" : DecisionTreeRegressor(),
    "Support vector regression" : SVR()}

# Словник, який має зберігати оцінку якості моделі
regression_results = {}

# Перебір всіх ключей словника (назв моделей)
for key in models.keys():
    # Навчання моделі на навчальній вибірці
    models[key].fit(X_train, Y_train)
    # Прогнозування результатів для тестової вибірки
    Y_predict = models[key].predict(X_test)
    # Запис результатів
    regression_results[key] = mean_absolute_error(Y_test,
Y_predict)

# Запис результатів до серії та демонстрація
results = pd.Series(regression_results)
print(results)

Linear regression          4.074763
K neighbors regressor      3.129121
Decision tree regressor    3.691884
Support vector regression  3.545011
dtype: float64

# Побудова та відображення графіка, що демонструє
# якість різних моделей регресії
results.plot(kind = "bar")
plt.show()
```

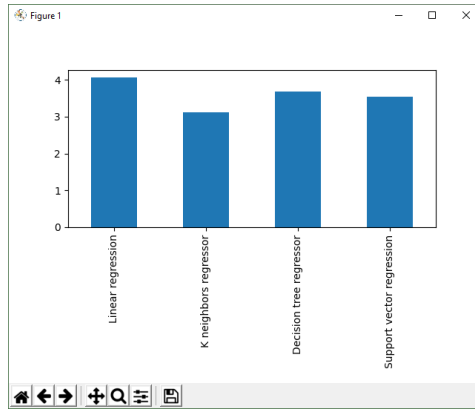


Рисунок 4.3 – Порівняння якості регресійних моделей за середньою абсолютною помилкою

4.3 Завдання на лабораторну роботу

1. Обрати в додатку В вибірку для регресійного аналізу. Конвертувати та, якщо необхідно, виконати попередню обробку даних.
2. Вирішити задачу регресії за допомогою програми WEKA:
 - завантажити вибірку (ARFF файл);
 - побудувати та оцінити моделі регресії за допомогою всіх розглянутих в лабораторній роботі алгоритмів в WEKA;
 - для метода M5P (model trees and regression trees) змінюючи параметри build regression tree, unpruned, useUnsmoothed досягти найліпшої якості прогнозування;
 - навести одну з отриманих регресійних моделей (рівняння регресії) та визначити які з атрибутів є найбільш значущими (на основі отриманих коефіцієнтів рівняння регресії) для прогнозування значень;
 - порівняти отримані моделі регресії за допомогою модуля Experimenter.
3. Вирішити задачу регресії за допомогою засобів мови програмування Python:
 - завантажити вибірку (CSV файл);
 - побудувати та оцінити моделі регресії за допомогою всіх розглянутих в лабораторній роботі алгоритмів бібліотеки scikit-learn;
 - змінюючи параметри налаштування деяких алгоритмів, досягти найліпшої якості прогнозування;

- визначити які з атрибутів є найбільш значущими (на основі отриманих коефіцієнтів рівняння регресії) для прогнозування значень цільового атрибута;

- порівняти отримані регресійні моделі та значимість атрибутів за допомогою графічних засобів бібліотеки `matplotlib`.

4. Відповісти на 5 контрольних запитань.

5. Оформити звіт з роботи.

4.4 Контрольні питання

1. Що таке задача регресії? Наведіть практичний приклад.

2. В чому схожість та відмінність між задачами регресії та класифікації?

3. Що таке навчання з учителем і без учителя? До якої стратегії відноситься задача регресії?

4. Які алгоритми побудови моделей регресії ви знаєте?

5. Опишіть один із методів побудови моделей регресії.

6. Які метрики якості регресійних моделей ви знаєте? Опишіть їх.

7. Як за допомогою регресійної моделі визначити вплив атрибутів на результат прогнозування цільової змінної?

8. Як побудувати й оцінити модель регресії в програмі WEKA?

9. Як порівняти якість декількох регресійних моделей в WEKA?

10. Опишіть принципи роботи з моделями регресії за допомогою бібліотеки `scikit-learn`?

11. Які метрики якості регресійних моделей представлено в бібліотеці `scikit-learn`?

12. Як порівняти якість декількох регресійних моделей за допомогою Python?

4.5 Зміст звіту

Звіт має складатися з таких розділів:

1. Тема і мета роботи;

2. Завдання до роботи;

3. Результати виконання завдань;

4. Відповіді на контрольні запитання;

5. Висновки, що відображають отримані результати роботи та їх критичний аналіз.

5 ЛАБОРАТОРНА РОБОТА № 5 ПОШУК АСОЦІАТИВНИХ ПРАВИЛ

5.1 Мета роботи

Ознайомитися та отримати навички роботи з програмою WEKA та бібліотеками мови програмування Python для пошуку асоціативних правил. На практиці вивчити роботу алгоритмів пошуку асоціативних правил та навчитися інтерпретувати результати їх роботи.

5.2 Основні теоретичні відомості

Асоціація – це виявлення закономірностей між пов’язаними подіями. Прикладом такої закономірності служить правило, яке вказує, що з події X слідує подія Y . Такі правила називаються асоціативними.

Пошук асоціативних правил – це метод машинного навчання заснованого на правилах для знаходження нетривіальних відношень між змінними у великих базах даних. Метою є ідентифікація сильних правил, які виявляються в базах даних з використанням деяких вимірів зацікавленості. Іншими словами, метою пошуку є встановлення таких залежностей: якщо в транзакції зустрівся деякий набір елементів X , то на підставі цього можна зробити висновок про те, що інший набір елементів Y також же повинен з’явитися в цій транзакції.

Вперше ця задача була запропонована для знаходження типових шаблонів покупок, що здійснюються в супермаркетах, тому іноді її ще називають аналізом ринкової кошика (market basket analysis).

Асоціативні правила ефективно використовуються в сегментації покупців по поведінці при покупках, аналізі переваг клієнтів, плануванні розташування товарів в супермаркетах, кросмаркетингу, адресній розсилці.

Однак, сфера застосування асоціативних правил не обмежується лише однією торгівлею. Їх також успішно застосовують і в інших областях: медицині, для аналізу відвідувань веб-сторінок (web mining), для аналізу тексту (text mining), для аналізу даних по перепису населення, в аналізі та прогнозуванні збоїв телекомунікаційного устаткування та інших.

Один з перших алгоритмів, що ефективно вирішує подібний клас задач — це алгоритм APriori.

5.2.1 Метрики асоціативних правил

Нехай є база даних, що складається з купівельних транзакцій. Кожна транзакція – набір товарів, куплених покупцем за один візит.

Нехай $I = \{i_1, i_2, \dots, i_n\}$ – це множина (набір) товарів (елементів), D – множина транзакцій, де кожна транзакція T — набір елементів з I , тобто $T \subseteq I$. Будемо говорити, що транзакція T містить X , деякий набір елементів з I , якщо $X \subset T$.

Асоціативним правилом називається імплікація $X \Rightarrow Y$, де $X \subset I$, $Y \subset I$ та $X \cap Y = \emptyset$.

Нехай N – кількість транзакцій, N_X – кількість транзакцій, які містять набір X , а $N_{X \cup Y}$ – кількість транзакцій, що містять і X , і Y .

Підтримка (support) правила – це частка транзакцій з D , що містить набір $X \cup Y$:

$$\text{supp}(X \Rightarrow Y) = \text{supp}(X \cup Y) = \frac{N_{X \cup Y}}{N}$$

Достовірність (confidence) правила вказує, яка ймовірність того, що з X слідує Y . Це відношення транзакцій з D , що містять X до транзакцій з D , що містять і X , і Y . Іншими словами це відношення підтримки всього правила до підтримки лівої частини правила:

$$\text{conf}(X \Rightarrow Y) = \frac{\text{supp}(X \Rightarrow Y)}{\text{supp}(X)} = \frac{N_{X \cup Y}}{N_X}$$

Параметр lift визначається діленням достовірності на підтримку правої частини правила:

$$\text{lift}(X \Rightarrow Y) = \frac{\text{supp}(X \Rightarrow Y)}{\text{supp}(X) * \text{supp}(Y)} = \frac{\text{conf}(X \Rightarrow Y)}{\text{supp}(Y)}$$

Параметр conviction дорівнює:

$$\text{conv}(X \Rightarrow Y) = \frac{1 - \text{supp}(Y)}{1 - \text{conf}(X \Rightarrow Y)}$$

Параметр leverage – це відношення додаткових екземплярів, що

покриваються правилом, до очікуваних в тому випадку, коли ліва і права частини правила були би статистично незалежними.

Наприклад, нехай ми маємо 1000 екземплярів в вибірці (N), при цьому ліва частина правила покриває 200 з них (N_X), права окремо покриває 100 (N_Y), а все правило покриває 50 ($N_{X \cup Y}$). Частка екземплярів, що покриваються правилом (підтримка) дорівнює $50 / 1000 = 0,05$. Кількість екземплярів, які були б покриті правилом у випадку, коли ліва і права частина правила незалежні, дорівнює $200 * 100 / 1000 = 20$. Параметр leverage для цього прикладу дорівнює $50 - 20 = 30$, що в пропорції від загальної вибірки дорівнює $30 / 1000 = 0,03$.

Алгоритми пошуку асоціативних правил призначені для знаходження всіх правил $X \Rightarrow Y$, причому підтримка і достовірність цих правил повинні бути вище деяких наперед визначених порогів, званих відповідно мінімальною підтримкою (minsupport) і мінімальною достовірністю (minconfidence).

Задача знаходження асоціативних правил розбивається на дві підзадачі:

- знаходження всіх наборів елементів, які задовольняють порогу minsupport (такі набори елементів називаються такими, що часто зустрічаються);
- генерація правил з наборів елементів, знайдених згідно з достовірністю, що задовольняє порогу minconfidence.

5.2.2 Робота з програмою WEKA

Для пошуку асоціативних правил за допомогою програми WEKA можна використати функції, що надає графічний інтерфейс користувача середовища Explorer, а саме: вкладку Associate (Асоціативні правила).

Після завантаження файлу з даними необхідно вибрати алгоритм, встановити необхідні налаштування та виконати пошук асоціативних правил. В результаті буде отримано список із заданої кількості асоціативних правил та оцінку їх якості (підтримка, достовірність тощо).

Більш детальний опис роботи з вкладкою Associate графічного інтерфейсу користувача програми WEKA наведено в додатку А.

Програма WEKA надає можливість виконання пошуку

асоціативних правил за такими алгоритмами:

- алгоритм Apriori;
- алгоритм FPGrowth.

Секція «Associator model» містить інформацію про побудовану модель.

Для алгоритму Apriori секція містить значення мінімальної підтримки, мінімальне значення обраної метрики (зазвичай достовірність), кількість циклів алгоритму. Далі розташовані знайдені часті набори даних та знайдені правила.

Для алгоритму FPGrowth буде вказана загальна кількість знайдених правил та задана кількість найліпших з них.

Для кожного з правил будуть відображені деякі числові параметри: число перед стрілкою вказує кількість екземплярів вибірки, для яких ліва умовна частина правила вірна (N_X), а число після стрілки вказує кількість екземплярів вибірки, для яких вірна ліва і права частини правила (N_{XUY}).

Розглянемо параметри налаштування використовуваних алгоритмів пошуку асоціативних правил в WEKA (табл. 5.1).

Таблиця 5.1 – Параметри налаштування алгоритмів

Метод	Параметр
Apriori	car – пошук класових (зі значенням цільового атрибута в правій частині) або звичайних асоціативних правил. classIndex – індекс цільового атрибута. Якщо встановлено значення -1, буде обраний останній атрибут. delta – ітеративно зменшувати значення порогу підтримки на дане значення. Зменшення буде відбуватися до тих пір, поки не буде досягнуто мінімальне значення підтримки чи не буде згенеровано задану кількість правил. lowerBoundMinSupport – нижня межа порогу підтримки. metricType – встановлює тип метрики, за якою будуть ранжуватися правила (confidence, lift, leverage, conviction). minMetric – мінімальне граничне значення для метрики. numRules – кількість правил, які необхідно знайти. outputItemSets – чи виводити часті набори. removeAllMissingCols – чи прибирати колонки (атрибути) в яких всі значення відсутні. significanceLevel – рівень значущості (для достовірності). upperBoundMinSupport – верхня межа мінімальної підтримки.

	Ітеративне зменшення підтримки починається з цього значення
FPGrowth	delta – ітеративно зменшувати значення порогу підтримки на дане значення. Зменшення буде відбуватися до тих пір, поки не буде досягнуто мінімальне значення підтримки чи не буде згенеровано задану кількість правил. findAllRulesForSupportLevel – знайти всі правила, які задовольняють нижній межі мінімального значення підтримки та мінімальному значенню метрики. Включення цього режиму скасує виконання ітеративного зменшення підтримки для знаходження заданого кількості правил. lowerBoundMinSupport – нижня межа порогу підтримки як частка кількості примірників. maxNumberOfItems – максимальна кількість примірників у частому наборі; значення -1 означає без обмежень. metricType – встановлює тип метрики, за якою будуть ранжуватися правила. minMetric – мінімальне граничне значення для метрики. numRulesToFind – кількість правил, які необхідно знайти. positiveIndex – встановлює індекс бінарного атрибуту, який буде розглядатися як позитивний. rulesMustContain – виводити правила, які містять задані об'єкти (список об'єктів, розділених комою). transactionsMustContain – для роботи алгоритму використовувати транзакції (примірники), які містять задані об'єкти. upperBoundMinSupport – верхня межа мінімальної підтримки. Ітеративне зменшення підтримки починається з цього значення. useORForMustContainList – – використовувати логічний зв'язок «або» замість «і» для списків обов'язкових елементів у транзакціях і правилах.

5.2.3 Робота з Python

Для пошуку асоціативних правил за допомогою мови програмування Python можна використовувати пакет mlxtend.

Mlxtend – це бібліотека Python з корисними інструментами для вирішення завдань із вивчення даних та машинного навчання. Ця бібліотека містить два необхідних алгоритма: Apriori та FP-Growth.

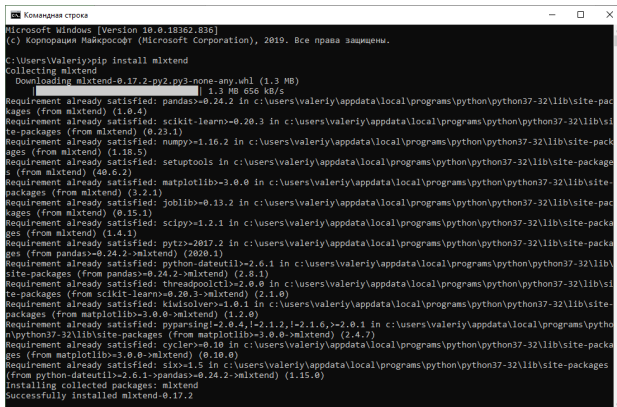
Для встановлення пакета mlxtend необхідно виконати таку

команду:

```
pip install mlxtend
```

В результаті виконання цієї команди (рис. 5.1) буде встановлено пакет mlxtend та бібліотеки від яких він залежить (якщо бібліотеки вже було встановлено, то повторної інсталяції не буде):

- mlxtend;
- pandas;
- scikit-learn;
- numpy;
- setuptools;
- matplotlib;
- joblib;
- scipy;
- pytz;
- python-dateutil;
- threadpoolctl;
- kiwisolver;
- pyparsing;
- cycler;
- six.



```

Microsoft Windows [Version 10.0.18362.836]
(c) Корпорація Майкрософт (Microsoft Corporation), 2019. Все права захищені.

C:\Users\Valeriy>pip install mlxtend
Collecting mlxtend
  Downloading mlxtend-0.17.2-py2.py3-none-any.whl (1.3 MB)
    |#####| 1.3 MB 656 KB/s
Requirement already satisfied: pandas>=0.24.2 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from mlxtend) (1.0.4)
Requirement already satisfied: scikit-learn>=0.20.3 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from mlxtend) (0.23.1)
Requirement already satisfied: numpy>=1.16.2 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from mlxtend) (1.18.5)
Requirement already satisfied: setuptools in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from mlxtend) (40.6.2)
Requirement already satisfied: matplotlib>=3.0.0 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from mlxtend) (3.2.1)
Requirement already satisfied: joblib>=0.13.2 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from mlxtend) (0.15.1)
Requirement already satisfied: scipy>=1.2.1 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from mlxtend) (1.4.1)
Requirement already satisfied: pytz>=2017.2 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from pandas>=0.24.2->mlxtend) (2020.1)
Requirement already satisfied: python-dateutil>=2.6.1 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from pandas>=0.24.2->mlxtend) (2.8.1)
Requirement already satisfied: threadpoolctl>=2.0.0 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from scikit-learn>=0.20.3->mlxtend) (2.1.0)
Requirement already satisfied: kiwisolver>=1.0.1 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (1.2.0)
Requirement already satisfied: pyparsing>=2.0.4, <2.4.2, >=2.1.6, <2.8.0 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (2.4.7)
Requirement already satisfied: cycler>=0.10 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from matplotlib>=3.0.0->mlxtend) (0.10.0)
Requirement already satisfied: six>=1.5 in c:\users\valeriy\appdata\local\programs\python\python37-32\lib\site-packages (from python-dateutil>=2.6.1->pandas>=0.24.2->mlxtend) (1.15.0)
Installing collected packages: mlxtend
Successfully installed mlxtend-0.17.2

```

Рисунок 5.1 – Встановлення пакета mlxtend

Класи та методи пакету mlxtend легко взаємодіють з типами даних pandas, а саме з DataFrame. Завантажимо дані в програму за допомогою засобів бібліотеки pandas:

```
import pandas as pd
data = pd.read_csv("D:\\FoodMart.csv")
print(data.head())
```

	Acetominifen	Anchovies	Aspirin	...	Wine	Yogurt
0	1.0	NaN	NaN	...	NaN	NaN
1	1.0	NaN	NaN	...	NaN	NaN
2	NaN	NaN	NaN	...	NaN	NaN
3	NaN	NaN	NaN	...	NaN	NaN
4	NaN	NaN	NaN	...	NaN	NaN

Особливістю алгоритмів Apriori та FP-Growth, що представлено в пакеті mlxtend є те, що замість набору транзакцій вхідними даними є DataFrame, в якому стовпці відповідають атрибутам, а рядки – транзакціям. Елементи кадру даних представляються бінарними значеннями (bool), що означають входить (True) чи не входить (False) i-й атрибут до j-ї транзакції.

Виконаємо попередню обробку даних перед використанням алгоритмів:

```
# Зміна всіх невідомих значень на False
data = data.fillna(False)

# Встановлення типу bool для кожної колонки. При цьому
# всі не False значення отримають значення True
for column in data.columns:
    data[column] = data[column].astype("bool")

print(data.head())
```

	Acetominifen	Anchovies	Aspirin	...	Wine	Yogurt
0	True	False	False	...	False	False
1	True	False	False	...	False	False
2	False	False	False	...	False	False
3	False	False	False	...	False	False
4	False	False	False	...	False	False

Розберемо роботу з Apriori. Apriori – це алгоритм глибинного аналізу даних щодо частих одиниць у множинах і машинного навчання щодо асоціативних правил, що застосовується переважно до баз даних транзакцій.

Перш за все необхідно імпортувати метод, який відповідає за використовуваний алгоритм:

```
from mlxtend.frequent_patterns import apriori
```

Запустимо алгоритм Apriori попередньо встановивши необхідні параметри та збережемо результат пошуків:

```
result = apriori(data, min_support=0.01, use_colnames=True)

print(result)
```

	support	itemsets
0	0.014575	(Acetominifen)
1	0.015515	(Anchovies)
2	0.031030	(Aspirin)
3	0.015515	('Auto)
4	0.013164	(Bagels)
..	...	...
126	0.011754	(Milk, Soup)
127	0.013634	(Nuts, Preserves)
128	0.010343	(Soup, Pizza)
129	0.011283	(Waffles, Soup)
130	0.013164	(Soup, Wine)

Інтерпретуємо результат за допомогою метода `association_rules`:

```
from mlxtend.frequent_patterns import association_rules

rules = association_rules(result, metric="confidence",
min_threshold=0.1)
```

В результаті інтерпретації отримано DataFrame, який зберігає antecedents (ліву частину правила, X), consequents (праву частину правила Y) та метрики оцінки кожного з асоціативних правил.

Виведемо на екран 3 правила з найбільшою достовірністю. Для цього відсортуємо правила за параметром `confidence` та зробимо зріз:

```
rules = rules.sort_values("confidence", ascending = False)

matrix = rules[["antecedents", "consequents", "confidence"]]
print(matrix.head(3))
```

	antecedents	consequents	confidence
7	(Cereal)	(Soup)	0.232000
0	(Batteries)	(Cheese)	0.210084
38	(Juice)	(Soup)	0.203540

Робота з алгоритмом FP-Growth аналогічна роботі з Apriori:

```
# Імпорт алгоритма fpgrowth
from mlxtend.frequent_patterns import fpgrowth
# Виконання алгоритма й запис результату
```

```

result = fpgrowth (data, min_support=0.01, use_colnames=True)

# Інтерпретація результатів
rules = association_rules(result, metric="confidence",
min_threshold=0.1)

# Сортуювання результатів за достовірністю
rules = rules.sort_values("confidence", ascending = False)

# Відображення найкращих асоціативних правил
matrix = rules[["antecedents", "consequents", "confidence"]]
print(matrix.head(7))

```

	antecedents	consequents	confidence
44	(Cereal)	(Soup)	0.232000
34	(Batteries)	(Cheese)	0.210084
19	(Juice)	(Soup)	0.203540
15	(Lightbulbs)	(Dried)	0.200000
20	(Juice)	(Cheese)	0.194690
8	(Waffles)	(Soup)	0.181818
17	(Muffins)	(Cookies)	0.181818

5.3 Завдання на лабораторну роботу

1. Виконати пошук асоціативних правил за допомогою програми WEKA:

- завантажити вибірку «vote.arff»;
 - запустити алгоритм пошуку асоціативних правил Apriori;
 - визначити які значення для порогу підтримки та достовірності було використано в побудованій моделі;
 - записати 10 найкращих знайдених правил та вказати для них значення підтримки та достовірності;
 - запустити алгоритм пошуку асоціативних правил FPGrowth;
 - порівняти списки десяти найкращих правил, отриманих двома алгоритмами. Пояснити відмінність в роботі двох алгоритмів;
 - запустити алгоритм Apriori задавши значення `car = true`.
- Описати які асоціативні правила були отримані та що знаходиться в правій частині знайдених асоціативних правил;
- вирішити задачу пошуку асоціативних правил для одного з наборів даних: 'bank-data.csv', 'marketbasket.arff'. Проаналізувати та пояснити знайдені правила;

2. Виконати пошук асоціативних правил за допомогою засобів мови програмування Python:

- конвертувати вибірку «marketbasket.arff» в CSV формат, та завантажити в програму;

- виконати пошук асоціативних правил за допомогою всіх розглянутих в лабораторній роботі алгоритмів бібліотеки mlxtend;

- використовуючи одну з розглянутих метрик, відобразити найкращі з отриманих асоціативних правил.

3. Відповісти на 5 контрольних запитань.

4. Оформити звіт з роботи.

5.4 Контрольні питання

1. Що таке задача пошуку асоціативних правил? Наведіть практичний приклад.

2. Що таке частий набір?

3. Що таке сильне асоціативне правило?

4. З яких двох кроків складається пошук асоціативних правил?

5. Які алгоритми пошуку асоціативних правил ви знаєте?

6. Опишіть один із методів пошуку асоціативних правил.

7. Як формуються правила зі знайдених частих наборів?

8. Які метрики асоціативних правил ви знаєте? Опишіть їх.

9. Як виконати пошук асоціативних правил за допомогою WEKA?

10. Для чого використовується пакет mlxtend?

11. Які функції бібліотеки mlxtend ви знаєте?

12. Опишіть принцип пошуку асоціативних правил за допомогою бібліотеки mlxtend?

5.5 Зміст звіту

Звіт має складатися з таких розділів:

1. Тема і мета роботи;

2. Завдання до роботи;

3. Результати виконання завдань;

4. Відповіді на контрольні запитання;

5. Висновки, що відображають отримані результати роботи та їх критичний аналіз.

6 ЛАБОРАТОРНА РОБОТА № 6 ПРОГРАМНІ ЗАСОБИ КЛАСТЕРИЗАЦІЇ ДАНИХ

6.1 Мета роботи

Ознайомитися та отримати навички роботи з програмою WEKA та бібліотеками мови програмування Python для вирішення задачі кластеризації. На практиці вивчити роботу алгоритмів кластеризації, навчитися інтерпретувати результати їх роботи та вибирати найкращий метод для розв'язуваної прикладної задачі.

6.2 Основні теоретичні відомості

Кластеризація – це угруповання об'єктів (спостережень, подій) на основі даних (властивостей), що описують сутність цих об'єктів, у кластери.

Задача кластеризації (кластерний аналіз) – це задача автоматичного розбиття елементів деякої множини на групи в залежності від їх подібності. Під час розбиття перевіряється набір «точок» та відбувається їх групування в «кластери» згідно якоїсь міри довжини. Під «точками» мається на увазі n -вимірний вектор характеристик.

Об'єкти всередині кластера повинні бути «схожими» один на одного і відрізнятися від об'єктів, що увійшли в інші кластери. Чим більше схожі об'єкти всередині кластера і чим більше відмінностей між кластерами, тим точніша кластеризація.

Кластеризація є однією із фундаментальних задач data mining. Список прикладних областей, де вона застосовується, широкий: сегментація зображень, маркетинг, боротьба з шахрайством, прогнозування, аналіз текстів тощо.

Кластеризація відноситься до стратегії навчання без учителя (англ. Unsupervised Learning), яке також називають неконтрольованим або некерованим навчанням.

На відміну від класифікації, де кожен об'єкт відноситься до однієї із заздалегідь визначених груп, кластеризація розбиває множину об'єктів на групи, які визначаються лише результатом роботи алгоритма.

Зазвичай процес вирішення задачі кластеризації складається з таких кроків: виділення характеристик, визначення метрики, розбиття об'єктів на групи та представлення результатів. Після отримання та

аналізу результатів можливе корегування обраної метрики та параметрів методу кластеризації до отримання оптимального результату.

Серед популярних алгоритмів кластеризації можна виділити такі:

- поділяючі методи кластеризації;
- ієрархічні методи кластеризації;
- імовірнісні методи кластеризації;
- методи засновані на щільності розташування об'єктів.

6.2.1 Робота з програмою WEKA

Для вирішення задачі кластеризації за допомогою програми WEKA можна використати функції, що надає графічний інтерфейс користувача середовища Explorer, а саме: вкладку Cluster (кластеризація).

Після завантаження файлу з даними й попередньої обробки необхідно вибрати алгоритм. Для обраного алгоритму необхідно встановити бажані параметри й почати кластеризацію. В результаті буде отримано основну статистичну інформацію, а саме: опис алгоритму, центроїди кластерів, графічне відображення та інше (в залежності від використаного метода кластерного аналізу).

Більш детальний опис роботи з вкладкою Cluster графічного інтерфейсу користувача програми WEKA наведено в додатку А

Програма WEKA надає можливість виконання кластерного аналізу за такими алгоритмами (у дужках наведено назву в програмі WEKA, при цьому перше слово перед крапкою означає тип алгоритму класифікації і також вказує назву папки, в якій знаходиться метод):

- поділяючий метод кластеризації К-середніх (SimpleKMeans);
- ієрархічний метод кластеризації (HierarchicalClusterer);
- імовірнісний метод кластеризації EM (EM);
- ієрархічний метод кластеризації COBWEB (COBWEB);
- метод заснований на щільності розташування об'єктів DBSCAN

(DBScan).

Для додання відсутніх методів в WEKA необхідно встановити пакет `optics_dbScan`.

Розглянемо параметри налаштування використовуваних алгоритмів кластеризації в WEKA (табл. 6.1).

Таблиця 6.1 – Параметри налаштування кластеризаторів

Метод	Параметр
-------	----------

SimpleKMeans	displayStdDevs – відображати значення стандартного відхилення для числових атрибутів і підрахунки для номінальних атрибутів. distanceFunction – функція відстані. dontReplaceMissingValues – не замінювати пропущені значення середнім значенням або модою. maxIterations – максимальна кількість ітерацій алгоритму. numClusters – кількість кластерів. preserveInstancesOrder – зберігати порядок примірників у вибірці. seed – випадковий сім для рандомізації вибірки.
Hierarchical Clusterer	distanceFunction – функція відстані. distanceIsBranchLength – у дендрограмі висота лінії, що зв'язує кластери, буде показувати відстань між ними. linkType – тип зв'язку для розрахунку відстані між двома кластерами. numClusters – кількість кластерів. printNewick – виводити кластери в форматі Newick.
EM	displayModelInOldFormat – використовувати старий формат представлення моделі (у випадках великої кількості кластерів). maxIterations – максимальна кількість ітерацій. minStdDev – мінімальне значення стандартного відхилення. numClusters – кількість кластерів (встановити значень -1 для автоматичного вибору кількості кластерів). seed – випадковий сім для рандомізації вибірки.
DBSCAN	database_Type – використовувана база даних. database_distanceType – функція відстані. epsilon – радіус пошуку. minPoints – мінімальна кількість об'єктів усередині радіуса.
COBWEB	acuity – мінімальне значення стандартного відхилення для числових атрибутів. cutoff – поріг до якого відсікати вузли дерева. saveInstanceData – зберегти інформацію про примірники для візуалізації. seed – випадковий сім для рандомізації вибірки.

Розглянемо результати роботи кластеризаторів в WEKA (Clusterer output).

Секція «Clustering model» відображає побудовану модель.

Для алгоритму SimpleKMeans ця секція буде містити кількість ітерацій алгоритму, загальну квадратичну помилку для всіх кластерів та центроїди побудованих кластерів. В ній також буде вказано застосований вид обробки порожніх значень атрибутів у об'єктів.

Для алгоритму EM буде вказана кількість кластерів, на які було розбито дані, кількість ітерацій алгоритму та центроїди побудованих кластерів.

Для алгоритму COBWEB буде вказана кількість об'єднань та розділів даних, кількість побудованих кластерів та текстове представлення побудованої ієрархії.

Для алгоритму DBSCAN ця секція буде містити налаштування алгоритму, кількість побудованих кластерів, віднесення кожного з об'єктів вибірки до конкретного кластеру чи до викидів.

Секція «Model and evaluation» містить інформацію про кількісний розподіл екземплярів по кластерах. При цьому буде вказано, скільки об'єктів було кластеризовано (Clustered Instances), а скільки не увійшли в жоден з кластерів (Unclustered instances).

Якщо було обрано опцію «Classes to clusters evaluation» (порівняння попередньої заданих класів з кластерами), то ця секція також буде містити результати оцінки якості кластеризації. Буде вказано, який з побудованих кластерів відповідає якому класу, буде побудовано матрицю помилок та вказана кількість невірно кластеризованих екземплярів.

6.2.2 Робота з Python

Для задачі кластеризації використаємо ті ж самі бібліотеки, як при вирішенні задачі класифікації.

В якості вибірки з даними візьмемо вже попередньо оброблений набір даних:

```
import pandas as pd
import matplotlib.pyplot as plt
data = pd.read_csv("D:\\iris.csv")

print(data.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
```

```
Data columns (total 5 columns):
#      Column      Non-Null Count  Dtype
---  -
0      sepalength  150 non-null    float64
1      sepalwidth  150 non-null    float64
2      petallength  150 non-null    float64
3      petalwidth  150 non-null    float64
4      class       150 non-null    object
dtypes: float64(4), object(1)
memory usage: 5.3+ KB
```

```
print(data.head(5))
```

```
      sepalength  sepalwidth  petallength  petalwidth      class
0           5.1         3.5         1.4         0.2  Iris-setosa
1           4.9         3.0         1.4         0.2  Iris-setosa
2           4.7         3.2         1.3         0.2  Iris-setosa
3           4.6         3.1         1.5         0.2  Iris-setosa
4           5.0         3.6         1.4         0.2  Iris-setosa
```

Перед початком кластеризації відокремимо цільовий атрибут, а для побудови кластеризатора будемо використовувати весь набір даних.

```
Y = data["class"]
X = data.drop(["class"], axis = 1)
```

Бібліотека `scikit-learn` надає багато класів з алгоритмами для вирішення задачі кластеризації.

Розберемо роботу з найбільш популярним алгоритмом кластеризації даних з `KMeans`. `KMeans` – це ітеративний алгоритм кластеризації, заснований на мінімізації сумарних квадратичних відхилень точок кластерів від центроїд (середніх координат) цих кластерів.

Перш за все необхідно імпортувати клас, який відповідає за використовуваний метод:

```
from sklearn.cluster import KMeans
```

Далі створюємо екземпляр цього класу й задаємо необхідні параметри (для `KMeans` це `n_clusters` – кількість кластерів, що формуються, а також кількість центроїдів для генерації):

```
км = KMeans(n_clusters = 3)
```

Для проведення кластеризації використаємо вже знайому функцію `fit()`:

```
KM.fit(X)
```

В результаті виконання цієї функції отримано кластеризатор, який зберігає центроїди виділених кластерів. Всі центроїди зберігаються в змінній `cluster_centers_` екземпляру класу `KMeans`:

```
print(KM.cluster_centers_)

array([[5.006      , 3.418      , 1.464      , 0.244      ],
       [6.85      , 3.07368421, 5.74210526, 2.07105263],
       [5.9016129 , 2.7483871 , 4.39354839, 1.43387097]])
```

За допомогою пакета `pandas` перетворимо дані в більш читабельний вигляд:

```
cluster_centers = pd.DataFrame(KM.cluster_centers_)
cluster_centers.columns = X.columns
print(cluster_centers)
```

	sepalength	sepalwidth	petallength	petalwidth
0	5.006000	3.418000	1.464000	0.244000
1	6.850000	3.073684	5.742105	2.071053
2	5.901613	2.748387	4.393548	1.433871

Кластеризатори, як і класифікатори можуть робити прогнози й зберігати результат в будь-яку змінну за допомогою функції `predict()`:

```
Y_predict = KM.predict(X)
```

Зауваження. Кластеризатор не знає про кількість та назви справжніх класів, а тому отримані кластери не мають назви, а просто пронумеровані.

Модуль `sklearn.metrics.cluster` містить різні метрики для оцінки побудованих кластеризаторів.

За допомогою функції `contingency_matrix()` можна порівняти попередньо задані класи з отриманими кластерами. В результаті буде побудовано адаптовану матрицю помилок:

```
from sklearn.metrics.cluster import contingency_matrix

print(contingency_matrix(Y, Y_predict))
[[50  0  0]
 [ 0 248]
 [ 0 36 14]]
```

Значення в стовпцях відповідають визначеним кластерам, а значення в рядках – значенням цільового атрибута. Тобто i, j елемент таблиці (де i – номер рядка, а j – номер стовпця) відповідає кількості

екземплярів i -го класу віднесених до j -го кластера.

За допомогою пакета `pandas` перетворимо дані в більш читабельний вигляд:

```
matrix = pd.DataFrame(contingency_matrix(Y, Y_predict))
matrix.index = Y.unique()

print(matrix)
```

	0	1	2
Iris-setosa	50	0	0
Iris-versicolor	0	2	48
Iris-virginica	0	36	14

Розрахуємо скільки екземплярів було правильно кластеризовано, знайдемо значення точності та помилки:

```
# Будемо співвідносити кластер до класу за найбільшою
# кількістю екземплярів класу в кластері
right_cluster_count = 0
for column in matrix.columns:
    right_cluster_count += matrix[column].max()
```

```
# Розраховуємо точність та помилку
accuracy = right_cluster_count / matrix.sum().sum()
error_rate = 1 - accuracy
```

```
# Демонструємо результати
print(right_cluster_count)
```

```
134
```

```
print(accuracy)
```

```
0.8933333333333333
```

```
print(error_rate)
```

```
0.10666666666666669
```

Розберемо роботу з методом DBSCAN, що заснований на щільності розташування об'єктів. DBSCAN (Density-Based Spatial Clustering of Applications with Noise) – популярний алгоритм кластеризації, який використовується в аналізі даних в якості однієї із замін методу k -середніх.

Метод не вимагає попередніх припущень про кількість кластерів,

але п'отрібно налаштувати два інших параметри: `eps` і `min_samples`. Дані параметри – це відповідно максимальна відстань між сусідніми точками й мінімальне число точок навколо (кількість сусідів), необхідних для утворення одного кластеру. Окрім номера кластера може бути значення -1. Воно означає, що екземпляр віднесено до шуму (немає класу, шум).

Приклад роботи з DBSCAN:

```
# Імпорт необхідних модулів
import numpy as np
from sklearn.cluster import DBSCAN

# Оголошення кластеризатора
DBSCAN_method = DBSCAN(eps = 0.5, min_samples = 5)
# Навчання кластеризатора прогнозування
Y_predict = DBSCAN_method.fit_predict(X)

# Побудова матриці
matrix = pd.DataFrame(contingency_matrix(Y, Y_predict))
matrix.index = Y.unique()
matrix.columns = np.unique(Y_predict)
print(matrix)
```

	-1	0	1
Iris-setosa	1	49	0
Iris-versicolor	6	0	44
Iris-virginica	10	0	40

Результати роботи деяких алгоритмів не тільки можна, а й варто візуалізовувати. Одним із таких алгоритмів є ієрархічна кластеризація, в процесі якої виконується побудова дендрограми.

Дендрограма – це деревоподібна діаграма, що містить рівні, що представляють собою кожен крок процесу послідовного укрупнення кластерів (дерево кластерів). За допомогою побудови дендрограми можна легко візуалізувати історію угруповань та визначити оптимальну кількість кластерів.

Побудуємо дендрограму (рис. 6.1) за допомогою функцій бібліотеки `scipy`, яка автоматично встановлюється з пакетом `scikit-learn`.

Імпортуємо необхідні методи з бібліотеки:

```
from scipy.cluster.hierarchy import linkage, dendrogram
```

Побудуємо дендрограму:

```
# Побудова матриці зв'язків
```

```

mergins = linkage(X, method = "complete")

# Побудова графіка ієрархічної кластеризації
# у вигляді дендрограми.
dendrogram(mergins, labels = Y.tolist())

# Відображення дендрограми
plt.show()

```

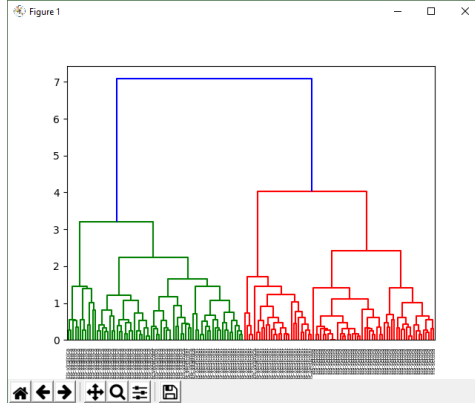


Рисунок 6.1 – Побудована дендрограма

6.3 Завдання на лабораторну роботу

1. Вирішити задачу кластеризації за допомогою програми WEKA:

- завантажити вибірку «bank.arff»;
- запустити алгоритм кластеризації SimpleKMeans, задаючи значення параметра K (кількість кластерів) від 1 до 12;
- записати в таблицю значення сум квадратичних помилок, одержуваних при різних значеннях. Проаналізувати результат;
- для значення K = 5 візуалізувати результати кластеризації (по осі абсцис відкласти назву (номер) кластера, по осі ординат – номер примірника в кластері);
- описати скільки примірників потрапило в кожен із кластерів, скільки ітерацій знадобилося для кластеризації даних;
- скласти таблицю з характеристиками центрів;
- завантажити вибірку «iris.arff»;
- запустити алгоритм кластеризації SimpleKMeans з K = 3 та

оцінити якість кластеризації, порівнюючи кластери з попередньо заданими класами;

- записати характеристики кожного центроїду;
- описати як співвідносяться кластери та значення цільового атрибуту, скільки екземплярів було віднесено до «невірних» кластерів, а який клас виявився «легким» для виділення;
- візуалізувати результати, використовуючи різні атрибути для осі ординат (при візуалізації екземпляри, позначені квадратами мають бути віднесені до «невірного» кластеру);
- визначити, на що впливає параметр «seed»;
- завантажити вибірку «flagdata.arff»;
- запустити алгоритм COBWEB з параметрами $C=0,4$ (0,35), `saveInstanceData = True`, `cluster mode = Use training set`;
- візуалізувати отриману дендрограму та вказати, що спільного у прапорів, які опинилися в одному кластері;
- завантажити вибірку «zoo.arff»;
- обрати з вибірки частину тварин на власний розсуд (за типом);
- запустити алгоритм Hierarchical Clusterer (тип тварини не використовувати в кластеризації, а назву за допомогою фільтру NominalToString перетворити на рядковий тип);
- візуалізувати результати роботи алгоритму та оцінити, чи є логічний сенс в створюваних кластерах;
- згенерувати вибірку за допомогою алгоритму BIRCHCluster. До вибірки згенерувати також флаг класу. Згенерований набір зберегти.
- за допомогою налаштувань методу DBSCAN досягнути найкращої кластеризації даних.

2. Вирішити задачу кластеризації за допомогою засобів мови програмування Python:

- конвертувати вибірку «vehicle.arff» в CSV формат, та завантажити в програму;
- виконати кластеризацію за допомогою алгоритму KMeans (кількість кластерів має дорівнювати кількості класів);
- продемонструвати та оцінити отримані результати;
- конвертувати вибірку «zoo.arff» в CSV формат та завантажити в програму;
- застосувати алгоритм ієрархічної кластеризації (при побудові матриці зв'язків видалити атрибути animal та type);
- візуалізувати отримані результати (дендрограма);
- конвертувати вибірку, яку було згенеровано за допомогою

- алгоритму BIRCHCluster у WEKA в CSV формат та завантажити;
- виконати кластеризацію за допомогою алгоритму DBSCAN;
 - продемонструвати та оцінити отримані результати.
3. Відповісти на 5 контрольних запитань.
 4. Оформити звіт з роботи.

6.4 Контрольні питання

1. Що таке задача кластеризації? Наведіть практичний приклад.
2. Що таке навчання з учителем і без учителя? До якої стратегії відноситься задача кластеризації?
3. В чому схожість та відмінність задач класифікації та кластеризації?
4. Чим визначається «схожість» об'єктів при вирішенні задачі кластеризації?
5. Які метрики застосовують для розрахунку відстані між екземплярами та кластерами?
6. Що таке однорівнева й ієрархічна кластеризація? Що таке чітка та нечітка кластеризація?
7. Які алгоритми кластеризації ви знаєте?
8. Опишіть один із методів кластеризації.
9. Як оцінити якість побудованої моделі для завдання кластеризації?
10. Опишіть принципи роботи при вирішенні задачі кластеризації за допомогою бібліотеки scikit-learn?

6.5 Зміст звіту

Звіт має складатися з таких розділів:

1. Тема і мета роботи;
2. Завдання до роботи;
3. Результати виконання завдань;
4. Відповіді на контрольні запитання;
5. Висновки, що відображають отримані результати роботи та їх критичний аналіз.

ПЕРЕЛІК ПОСИЛАНЬ

1. Олійник А.О. Інтелектуальний аналіз даних: навчальний посібник / А. О. Олійник, С. О. Субботін, О. О. Олійник. – Запоріжжя: ЗНТУ, 2012. – 278 с.
2. Witten, I. H. Data mining: practical machine learning tools and techniques.–3rd ed. / Ian H. Witten, Frank Eibe, Mark A. Hall. – Morgan Kaufmann Publishers, 2011. – 629 p.
3. Han J. Data Mining: Concepts and Techniques (Second Edition) / J. Han, M. Kamber – Morgan Kaufmann Publishers, 2006. – 800 p.
4. Weka 3: Data Mining [Electronic resource]. – Access mode: <http://www.cs.waikato.ac.nz/ml/weka>
5. Welcome to Python.org [Electronic resource]. – Access mode: <https://www.python.org>
6. Pandas – Python Data Analysis Library [Electronic resource]. – Access mode: <https://pandas.pydata.org>
7. Matplotlib: Python plotting [Electronic resource]. – Access mode: <https://matplotlib.org>
8. Scikit-learn: machine learning in Python [Electronic resource]. – Access mode: <https://scikit-learn.org/stable>
9. Home – mlxtend [Electronic resource]. – Access mode: <http://rasbt.github.io/mlxtend>
10. UCI Machine Learning Repository [Electronic resource]. – Access mode: <https://archive.ics.uci.edu/ml/index.php>

ДОДАТОК А

ІНТЕРФЕЙС ПРОГРАМИ WEKA

Для того, щоб вдало використовувати програму необхідно зрозуміти, як з нею працювати. Для цього розглянемо можливості графічного інтерфейсу користувача.

А.1 Головне вікно програми

Основне вікно програми – це Weka GUI Chooser (рис. А.1). Опишемо більш докладно призначення керуючих елементів даного вікна.

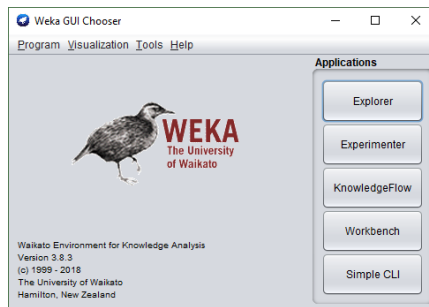


Рисунок А.1 – Головне вікно програми WEKA

Основне вікно програми надає доступ до чотирьох модулів програми:

- Explorer – середовище для дослідження даних;
- Experimenter – середовище для проведення порівняльного аналізу роботи різних алгоритмів при обробці наборів даних;
- KnowledgeFlow – середовище, що підтримує таку ж функціональність, як і Explorer, але із застосуванням графічного представлення потоків даних;
- Workbench – середовище, яке об'єднує всі три наведені вище середовища в одне з можливістю перемикавання подання;
- Simple CLI – командний інтерфейс для безпосереднього виконання команд WEKA.

Головне меню програми складається з чотирьох пунктів:

1. Program:

- LogWindow – відкриває вікно логів, яке зберігає всю

інформацію, виведену в потоки stdout або stderr;

- Memory usage – використання пам'яті;
- Settings – налаштування інтерфейсу;
- Exit – вихід.

11. Tools:

- Package manager – встановлення додаткових пакетів, що містять додаткові алгоритми, засоби візуалізації тощо;
- ArffViewer – редактор arff-файлів;
- SqlViewer – модуль перегляду баз даних;
- Bayes net editor – модуль для редагування, візуалізації та навчання Байєсови мереж (Bayes nets).

12. Visualization – засоби візуалізації даних WEKA.

- Plot – відображення 2D-графіка набору даних;
- ROC – відображає раніше збережену ROC-криву;
- TreeVisualizer – відображає спрямовані графи, тобто дерева рішень;

- GraphVisualizer – візуалізує графіку для Байєсових мереж;
- BoundaryVisualizer – дозволяє візуалізувати границі рішень класифікаторів у двох вимірах.

13. Help – розділ довідкової інформації.

- Weka homepage – домашня сторінка проекту WEKA;
- HOWTOs, code snippets, etc. – приклади, що стосуються розробки та використання WEKA;
- Weka on Sourceforge – сторінка проекту WEKA на Sourceforge.net;
- SystemInfo – значення деяких змінних середовища Java/WEKA.

A.2 Модуль Explorer

Це основний модуль програми, який дозволяє завантажити і попередньо обробити дані (вкладка Preprocess), вирішити задачу класифікації або регресії (Classify), кластеризації (Cluster), пошуку асоціативних правил (Associate), відбору атрибутів (Select Attributes) і візуалізації (Visualize).

Кожна задача має свою вкладку в загальному вікні. Спочатку доступна тільки вкладка Preprocess, оскільки для виконання інших завдань потрібні дані. Значимо, що послідовність вкладок не завжди відповідає етапам вирішення задачі. Наприклад, після завантаження

даних можна перейти до відбору атрибутів.

Внизу кожної вкладки відображається рядок статусу. Клік правою кнопкою миші на рядку статусу видає контекстне меню:

- Memory information – кількість доступної пам'яті;

- Run garbage collector – запуск збирача сміття Java, що очищає області пам'яті, які більше не використовуються, дозволяючи звільнити пам'ять для нових завдань.

Кнопка «LOG» дозволяє побачити лог подій що відбулися за час роботи WEKA.

У правій частині статусного рядка зображена пташка Ківі. Якщо вона рухається, то програма робить обчислення, якщо сидить нерухомо, то програма знаходиться в режимі очікування. Після значка «X» відображається кількість запущених процесів.

A.2.1 Завантаження і попередня обробка даних (Preprocess)

Спочатку на першій вкладці «Preprocess» вгорі вікна активні чотири кнопки, які дозволяють завантажити дані з файлу (Open file), з віддаленого джерела (Open URL), з бази даних (Open DB) або згенерувати модельні дані (Generate). Найчастіше доводиться користуватися даними з файлу.

Після завантаження файлу на панелі попередньої обробки даних з'являється інформація про дані (рис. A.2).

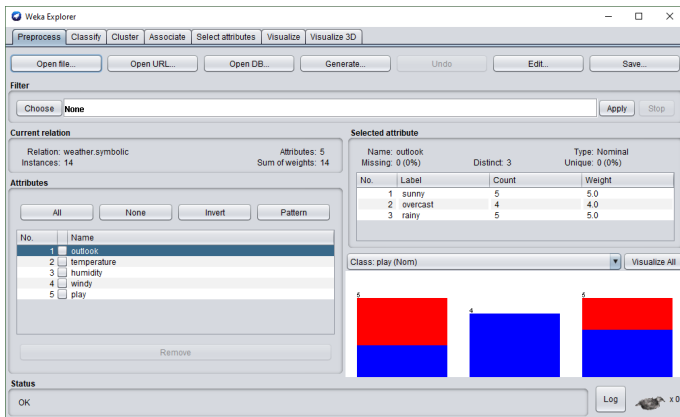


Рисунок A.2 – Вкладка завантаження та попередньої обробки даних

Натискання на кнопку «Edit» дозволяє редагувати вихідні дані у табличній формі (з'являється вікно «Viewer»).

Панель «Current relation» містить 3 значення:

- Relation – назва відношення (задачі), яка була зазначена у файлі (застосування фільтрів може змінювати цю назву);
- Instances – кількість екземплярів (об'єктів);
- Attributes – кількість ознак (атрибутів).

Нижче знаходиться панель «Attributes» (атрибути чи іншими словами ознаки об'єктів вибірки даних). На ній знаходяться чотири кнопки і список атрибутів. Атрибути розташовані в тому порядку, в якому вони визначені в файлі даних. Список містить три колонки:

- No – номер, що ідентифікує атрибут та використовується для вказівки на атрибут в фільтрах (attributeIndex);
- Selection tick boxes – дозволяє вибрати атрибути для подальших операцій (як правило для видалення);
- Name – ім'я атрибута, що визначено у файлі даних.

При виборі одного з атрибутів, його ім'я підсвічується в списку синім кольором. При цьому змінюється вміст правої панелі «Selected attribute»:

- Name – ім'я атрибута;
- Type – тип атрибута, наприклад, Nominal або Numeric;
- Missing – кількість і відсоток примірників, для яких значення цього атрибута втрачено (не визначено);
- Distinct – кількість різних значень атрибута;
- Unique – кількість і відсоток примірників, для яких значення атрибута має унікальне значення (такого значення немає у жодного іншого екземпляра).

Нижче цієї статистики відображається інформація про всі значення даного атрибута.

Якщо атрибут має тип nominal (номінальний або категоріальний, значення з певної множини), список містить усі можливі значення атрибута з кількістю їх появи. Якщо атрибут відноситься до numeric (числовий), то список надає таку статистичну інформацію: мінімум, максимум, математичне очікування і стандартне відхилення.

Нижче представлена кольорова гістограма значень атрибута, яка залежно від атрибута, обраного в якості цільового (що визначає клас), показує розподіл значень атрибута. Можна подивитися гістограми всіх атрибутів натисканням кнопки «Visualize all».

Вище списку всіх атрибутів знаходяться кнопки, що дозволяють вибирати атрибути (за допомогою чекбоксів).

Обрані таким чином атрибути можна видалити кнопкою «Remove».

Вкладка попередньої обробки даних дозволяє за допомогою фільтрів відібрати і трансформувати дані в необхідний формат.

Зліва на панелі «Filter» знаходиться кнопка вибору фільтра «Choose». За її допомогою можна вибрати один з попередньо визначених фільтрів. Після вибору фільтра з ієрархічного списку його назва з'явиться праворуч від кнопки «Choose». Клацнувши по назві фільтра, викликається діалогове вікно налаштування параметрів фільтра (The GenericObjectEditor Dialog Box). Це вікно надає інформацію про призначення фільтра та дозволяє налаштувати його параметри. Всі поля даного вікна мають спливаючі підказки.

Таке ж діалогове вікно використовується при налаштуванні інших об'єктів програми WEKA (наприклад, класифікаторів).

Внизу вікна налаштувань знаходяться 4 кнопки. Перші дві, «Open...» та «Save...», дозволяють зберегти налаштування об'єкта для майбутнього використання. Кнопка «Cancel» дозволяє повернутися без збереження проведених змін. По натисканню кнопки «OK» зміни зберігаються, і користувач повертається в «Explorer window».

По натисканню на кнопку «Apply» у правій частині панелі «Filter» відбувається фільтрація (попередня обробка) вихідних даних відповідно до обраного алгоритму. Кнопка «Undo» призначена для скасування проведених над даними операцій. Всі зміни, що вносяться в дані в програмі, виконуються в оперативній пам'яті та не впливають на початковий файл. Для збереження змінених даних в новий файл призначена кнопка «Save».

Зверніть увагу, що деякі фільтри під час своєї роботи приймають в якості параметру атрибут класу, що задається за допомогою випадального списку над гістограмою. Так, наприклад, «supervised filters» вимагають, щоб клас був заданий, в той час, коли «unsupervised attribute filters» не приймають даний параметр до уваги.

A.2.2 Класифікація (Classify)

Вкладка «Classify» дозволяє вирішувати задачу класифікації для завантаженого набору даних (рис. A.3).

Вгорі вкладки знаходиться панель вибору класифікатора «Classifier».

Налаштування параметрів подібне вибору фільтра попередньої обробки даних.

На панелі «Test options» визначається метод тестування навченого класифікатора:

- на навчальній вибірці (use training set);
- на тестовій вибірці з окремого файлу (supplied test set);
- по блоках (cross-validation);
- за допомогою відсоткового розподілу початкової вибірки на навчання та контроль (percentage split).

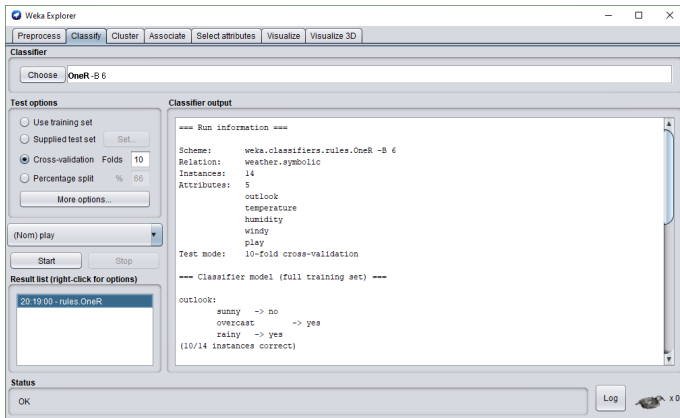


Рисунок А.3 – Вкладка рішення задачі класифікації

При виборі деяких опцій доведеться вказати параметри тестування. Наприклад, при виборі «cross-validation» треба вказати, на скільки блоків (фолдів, складок) розбивати вибірку. Наприклад, якщо вказати 10 фолдів, то вибірка буде розділена на 10 рівних частин, потім 9 частин буде використано для навчання і 1 для тестування. Процес буде повторений 10 разів, а результати усереднені.

Leave-one-out – це спеціальний випадок крос-перевірки, при якому кількість фолдів дорівнює кількості примірників у вибірці. Таким чином, кожен раз тільки один примірник виступає в якості тестової вибірки.

Кнопка «More options» дозволяє вибрати вид звіту про навчання класифікатора. Розглянемо параметри, які можна задати:

- Output model – відображення побудованої моделі;
- Output per-class stats – відображення статистики точність / ефективність та істина / неправда для кожного класу;
- Output entropy evaluation measures – відображення оцінки ентропії на критеріях;
- Output confusion matrix – відображення матриці неточностей передбачення класифікатора;
- Store predictions for visualization – передбачення класифікатора згруповані таким чином, щоб їх можна було візуалізувати;
- Output predictions – відображення передбачених значень класів для тестової вибірки;
- Output additional attributes – використовується у разі якщо разом з передбаченими значеннями класів необхідно вивести додаткові атрибути об'єктів (наприклад, якщо необхідно вивести ID атрибута для визначення неправильно класифікованих об'єктів);
- Cost-sensitive evaluation – помилка класифікації визначається з урахуванням матриці цінності;
- Random seed for xval/% Split – визначає випадковий сід (seed), який використовується при рандомізації (перетасуванні) даних перед їх поділом;
- Preserve order for % Split – скасовує рандомізацію даних перед їх поділом на навчальну і тестову вибірку;
- Output source code – вивід побудованої моделі у вигляді вихідного коду мовою Java, якщо це можливо.

У WEKA класифікатори спроектовані на прогнозування єдиного «класу», який є цільовим атрибутом в задачі класифікації. Деякі класифікатори можуть бути навчені для передбачення категоріальних (nominal) класів (класифікація), інші можуть бути навчені тільки для передбачення числових класів (задача регресії), а є класифікатори, що вирішують обидва типи задач. За замовчуванням у якості цільового обирається останній атрибут у списку. При необхідності цільовий атрибут можна змінити на панелі «Test options».

Процес навчання класифікатора починається натисканням на кнопку «Start». Поки класифікатор зайнятий навчанням, маленька пташка Ківі в рядку статусу рухається. Процес навчання можна зупинити в будь-який момент натисканням кнопки «Stop».

Коли процес навчання закінчений, праворуч на панелі «Classifier output» з'являється інформація з результатами навчання і тестування, а

також заповнюється панель «Result list».

Текст на панелі «Classifier output» містить інформацію, розбиту на декілька секцій. Розглянемо кожен з них.

Секція «Run information» містить параметри схеми навчання, ім'я відношення, екземпляри, атрибути і параметри тестування.

Секція «Classifier model (full training set)» містить текстове представлення моделі класифікації, отриманої в ході навчання.

Результати обраної стратегії навчання розбиті на такі секції:

- Summary – статистична інформація, що підводить підсумок точності класифікації для перевіірочних примірників;

- Detailed Accuracy By Class – більш детальна інформація по кожному класу;

- Confusion Matrix – матриця помилок моделі, елементи якої показують кількість тестових екземплярів, чий істинний клас є рядком і чий передбачений клас є колонкою.

Після навчання декількох класифікаторів панель «Result List» міститиме кілька записів. Клік лівою кнопкою миші на записі дозволяє переміщатися між результатами аналізу. Праве клацання на записі викликає контекстне меню з таким змістом:

- View in main window – показати результати в головному вікні (рівноцінно кліку лівою кнопкою миші);

- View in separate window – відкриває незалежне вікно з результатами;

- Save result buffer – виводить діалогове вікно, що дозволяє зберегти результати в текстовому файлі;

- Load model – завантажує попередньо навчену модель з бінарного файлу;

- Save model – зберігає навчену модель в бінарний файл у вигляді серіалізованого java об'єкта;

- Re-evaluate model on current test set – тестує ефективність навченої моделі на наборі даних, завантаженому за допомогою пункту «Set» на панелі «Test option»;

- Visualize classifier errors – виводить вікно з графіком, що візуалізує результати класифікації: вірно класифіковані примірники позначаються хрестиками, а невірно класифіковані – квадратами;

- Visualize tree / graph – надає графічне представлення структури класифікаційної моделі, якщо таке можливо (наприклад, для дерев рішень);

- Visualize margin curve – генерує графік, який ілюструє границі передбачення;
- Visualize threshold curve – генерує графік, який ілюструє різну ефективність передбачення, отриману шляхом варіювання порогового значення параметра класифікації;
- Visualize cost curve – генерує графік, який дає явне уявлення про очікувані витрати.
- Опції неактивні, якщо вони зараз не можуть бути застосовані.

A.2.3 Кластеризація (Cluster)

Вкладка «Cluster» містить схему навчання кластеризаторів (рис. A.4). Вибір функції кластеризації та налаштування її параметрів відбуваються таким же чином, як і для розглянутої вище задачі класифікації.

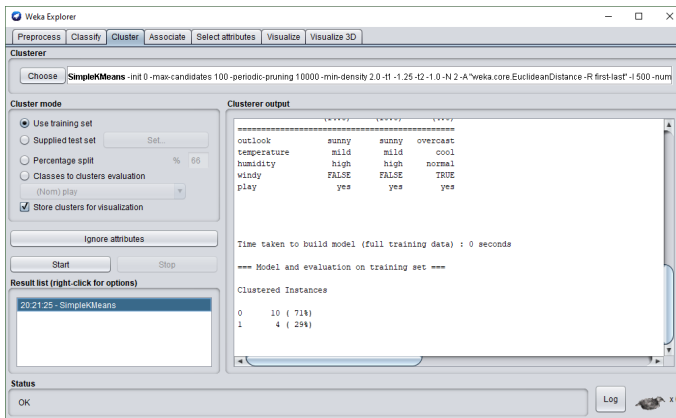


Рисунок A.4 – Вкладка рішення задачі кластеризації

Панель «Cluster mode» використовується для того, щоб визначити, що кластеризувати і як оцінювати результати. Перші три опції такі ж, як і для задачі класифікації: Use training set, Supplied test set и Percentage split (при цьому дані використовуються для віднесення до кластеру, а не для передбачення певного класу).

Четвертий метод «Classes to clusters evaluation» оцінює наскільки добре був обраний кластер, порівнюючи його з попередньо заданим класом в даних.

Додаткова опція у вигляді чекбокса «Store clusters for visualization» визначає, чи можливо буде візуалізувати кластери по закінченні навчання. При вирішенні завдань з дуже великими обсягами даних слід відключити дану опцію, щоб уникнути проблем з нестачею пам'яті.

Кнопка «Ignoring Attributes» дозволяє визначити, які атрибути слід ігнорувати при проведенні кластеризації.

Вкладка Кластеризації також як і вкладка класифікації містить кнопки «Start / Stop», панель результатів і список результатів. Клацання правою кнопкою миші на запису у списку результатів дає контекстне меню з двома опціями візуалізації: Visualize cluster assignments и Visualize tree.

A.2.4 Асоціативні правила (Associate)

Вкладка «Associate» містить схему навчання асоціативних правил (рис. А.5). Алгоритми пошуку асоціативних правил обираються, налаштовуються і виконуються так, як описано вище.

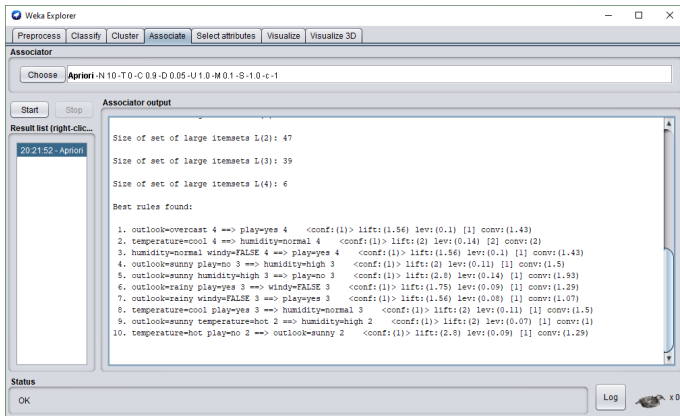


Рисунок А.5 – Вкладка для пошуку асоціативних правил

A.2.5 Відбір атрибутів (Select attributes)

Відбір атрибутів включає перебір всіх можливих комбінацій атрибутів даних для пошуку підмножини атрибутів, що дають найкращий результат передбачення (рис. А.6).

Для цього повинні бути налаштовані два об'єкти: оцінювач атрибутів і метод пошуку. Оцінювач (Attribute Evaluator) визначає який метод використовується для призначення значущості кожної підмножини атрибутів, а метод пошуку (Search Method) визначає стиль пошуку підмножин.

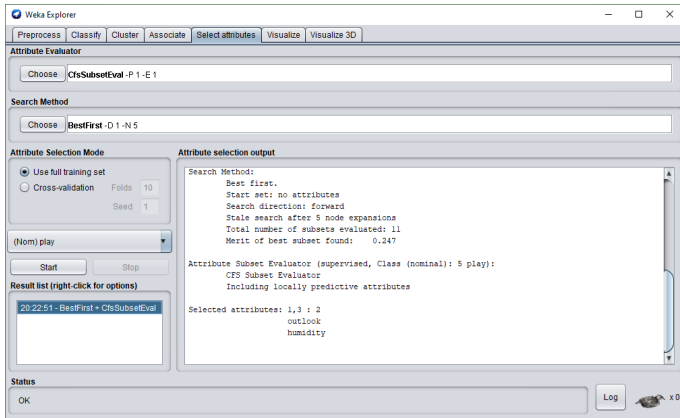


Рисунок А.6 – Вкладка вирішення задачі відбору атрибутів

Панель «Attribute Selection Mode» має два параметри:

- Use full training set – значимість підмножини атрибутів визначається для повного набору навчальних даних;
- Cross-validation (ковзний контроль, крос-перевірка) – значимість підмножини атрибутів визначається за допомогою кросперевірки. Поля Fold і Seed визначають кількість блоків (folds) і випадковий сид (seed), використовуваний при перетасування даних.

Знизу знаходиться список, який задає цільовий атрибут, який буде використовуватися в якості класу.

По натисканню на кнопку «Start» запускається процес відбору атрибутів. Коли процес закінчено, результати виводяться на панель результатів «Attribute selection output» та додаються до списку результатів. Натискання правої кнопки миші на результати видає контекстне меню. Перші три пункти цього меню (View in main window, View in separate window та Save result buffer) такі ж як і для вкладки класифікації. Додатковими є Visualize reduced data, або якщо був обраний метод Principal-Components, то Visualize transformed data.

А.2.6 Візуалізація (Visualize)

Вкладка візуалізації дозволяє представити вихідні дані в графічному вигляді (рис. А.7). При відкритті вкладки візуалізації даних відображається діаграма розкиду даних для всіх атрибутів з кольоровим кодуванням згідно обраного класу. Розміри кожної з діаграм можуть бути змінені, можуть бути змінені розміри точок. У дані можна додати шум (jitter) для виявлення слабких точок. Кожен графік також можна відкрити в окремому вікні натисканням на нього.

Для застосування внесених змін та оновлення інформації на графіках необхідно натиснути кнопку «Update».

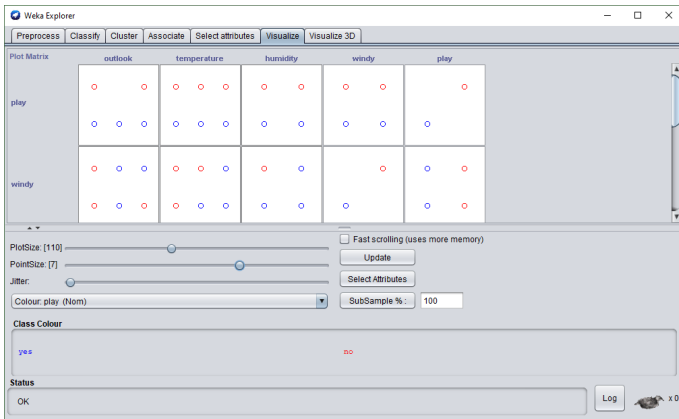


Рисунок А.7 – Вкладка візуалізації вихідних даних

На кожному графіку можна вибрати окремі точки за допомогою випадаючого списку «Select Instance» і зберегти їх в окремий файл.

А.3 Модуль Experimenter

Модуль Experimenter дозволяє проводити експерименти: запускати декілька різних алгоритмів для однієї або декількох задач та отримувати зведений звіт.

На вкладці «Setup» (рис. А.8) спочатку активні дві кнопки: «Open» (відкрити файл експерименту) і «New» (створити новий експеримент), а також дві опції списку параметрів експерименту: «Simple» (простий), «Advanced» (складний).

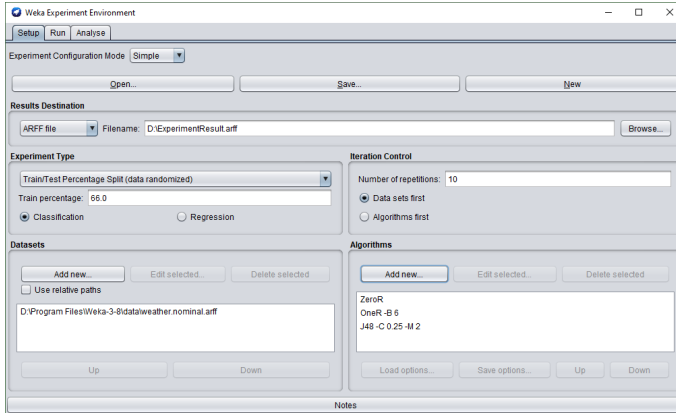


Рисунок А.8 – Установка параметрів експерименту

На панелі «Experiment Type» обирається тип експерименту:

- Cross-validation (default) – проводити верифікацію методом контролю по блоках;

- Train / Test Percentage Split (data randomized) – відсотковий поділ вибірки на контроль та навчання з випадковим порядком проходження примірників;

- Train / Test Percentage Split (order preserved) – відсотковий поділ вибірки із збереженням порядку (наприклад, коли в одному файлі зберігається і навчальна і тестова вибірка).

На панелі «Iteration Control» обирається число ітерацій (повторень експериментів). При повтореннях відбуваються нові розбиття вибірки на навчання і перевірку. Також тут вказується порядок проведення експерименту: перебирати спочатку всі задачі або всі алгоритми. На панелі «Datasets» можна обрати набори даних, а на панелі «Algorithms» – алгоритми (параметри їх вибираються аналогічно тому, як це робилося в модулі «Explorer»).

На вкладці «Run» присутня єдина кнопка «Start», що запускає експеримент на виконання.

Вкладка «Analyze» (рис. А.9) дозволяє проаналізувати результати експериментів. На панелі «Source» обирається, який експеримент аналізувати (тільки що проведений чи записаний у вихідному файлі).

На панелі «Configure test» визначається вид статистики для аналізу. Кнопка «Row» дозволяє вибрати дані, що будуть записані по рядках виведеної матриці (для прикладу виберемо з випадючого

списку Dataset), а кнопка «Column» – що буде записано за стовпцями (виберемо Scheme). У полі «Comparison field» вибирається тип даних, якими буде заповнена таблиця (вибір Percent_correct забезпечує заповнення відсотками вірної класифікації).

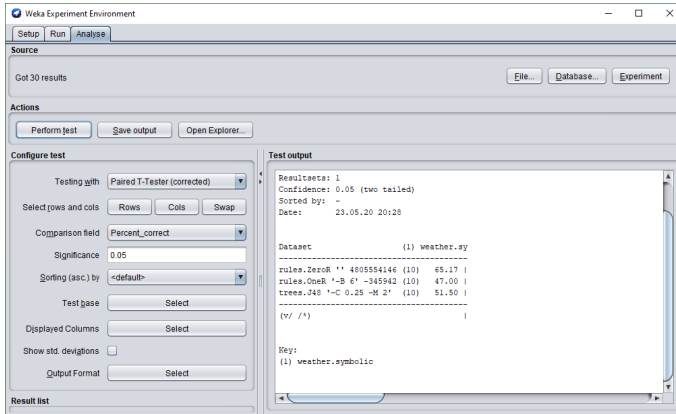


Рисунок А.9 – Оцінка результатів експерименту

Змінюючи значення що відображаються в Rows та Cols можна змінювати інформацію, що відображається в результуючій таблиці. Для генерації звіту натисніть кнопку «Perform test».

А.4 Модуль Knowledge flow

Інтерфейс KnowledgeFlow відображає концепцію потоків даних (рис. А.10).

Спочатку з примітивів різних видів будується весь шлях, яким мають пройти дані, від джерел до виходів. Зазвичай він такий: джерело даних (DataSources) – фільтр (Filter) (необов'язковий) – розбиття на навчальну і тестову множину (Evaluation) – класифікатор (Classifiers) чи кластеризатор (Clusterers) чи асоціатор (Associators)) – оцінка (Evaluation) – виведення даних (DataSinks).

Практично до будь-якого етапу можна приєднати відображення (Visualization).

Примітиви перетягуються в робоче поле з відповідних вкладок (клік на примітив – клік на робоче поле). Входи і виходи примітивів з'єднуються зв'язками, причому не можна з'єднати різнотипні за

даними виходи і входи. У примітиву може бути кілька виходів з різними даними. Зв'язки теж бувають різні, залежно від того, які дані по них передаються (з якого виходу вони йдуть). Зв'язуються примітиви шляхом вибору в контекстному меню, що з'являється при натисканні правою кнопкою миші на примітиві в робочому полі, у підпункті меню Connection певного виходу примітиву і наступного кліку на примітив, з яким слід встановити зв'язок. Всі примітиви, доступні для зв'язку, при цьому виділяються.

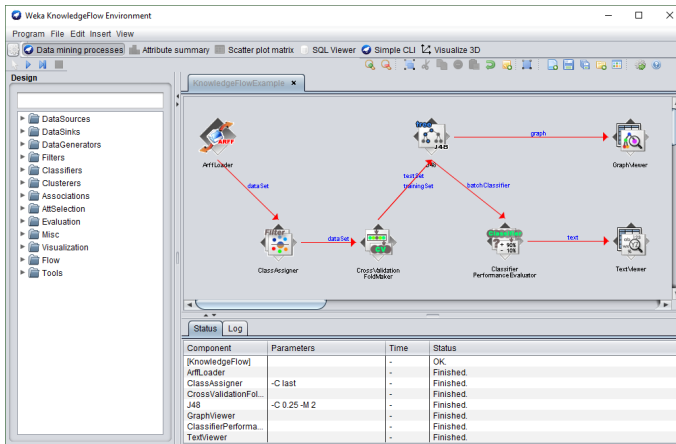


Рисунок А.10 – Модуль Knowledge flow

Після прокладки шляху за допомогою джерела даних завантажують дані, і вони автоматично проходять побудований шлях.

А.5 Модуль Workbench та Simple CLI

Модуль Workbench виконує роль середовища, яке об'єднує модулі Explorer, Experimenter та KnowledgeFlow в одне середовище з можливістю перемикавання подання.

Модуль Simple CLI виконує роль командного інтерфейсу для безпосереднього виконання команд WEKA. Він надає повний доступ до всіх можливостей програми Weka, а саме: до класифікаторів, фільтрів, кластерів.

Більш детальну інформацію про ці модулі можна знайти на офіційному сайті в розділі документації до програми WEKA.

ДОДАТОК В

ВАРІАНТИ ІНДИВІДУАЛЬНИХ ЗАВДАНЬ

Обрати з таблиці за номером варіанту $((N \bmod 16) + 1)$ з журналу набір даних для дослідження. Дослідити поставлену задачу, характеристики набору даних (атрибути), за необхідності провести попередню обробку даних та зменшити кількість об'єктів у вибірці, виділити аномалії та викиди, обрати стратегію роботи з об'єктами та пропусками, визначити стратегію тестування навчених алгоритмів. Для кожного з алгоритмів провести дослідження їх роботи на поставленій задачі, змінюючи параметри налаштування алгоритму.

Таблиця В.1 – Набори даних для задачі класифікації

1	adult.arff	9	mushroom.arff
2	bank-data.arff	10	postoperative.arff
3	breast-cancer.arff	11	spambase.arff
4	breast-w.arff	12	tic-tac-toe.arff
5	credit.arff	13	vehicle.arff
6	diabetes.arff	14	vote.arff
7	heart-statlog.arff	15	wine.arff
8	labor.arff	16	zoo.arff

Для вирішення задачі регресії обрати одну задачу за номером варіанту $((N \bmod 7) + 1)$.

Таблиця В.2 – Набори даних для задачі регресії

1	auto93.arff	5	cpu.arff
2	auto-mpg.arff	6	fishcatch.arff
3	autoprice.arff	7	housing.arff
4	bodyfat.arff		

В разі відсутності файлів даних, їх можна знайти на сайті UCI Machine Learning Repository (<https://archive.ics.uci.edu/ml/index.php>).